

Complete Course: Natural Language Processing (NLP)

From Text Processing to Transformers and Beyond

AI and Language Technology Mastery

Contents

1	Introduction to Natural Language Processing	3
1.1	What is NLP?	3
1.2	Applications of NLP	3
1.3	The NLP Pipeline	3
2	Text Preprocessing	4
2.1	Tokenization	4
2.2	Text Normalization	4
2.3	Stop Words Removal	6
2.4	Part-of-Speech Tagging	6
3	Text Representation	8
3.1	One-Hot Encoding	8
3.2	Bag of Words (BoW)	9
3.3	Term Frequency-Inverse Document Frequency (TF-IDF)	10
3.4	Word Embeddings	11
3.4.1	Word2Vec	12
3.4.2	GloVe (Global Vectors for Word Representation)	14
3.4.3	FastText	16
4	Sequence Models for NLP	19
4.1	Recurrent Neural Networks (RNNs)	19
4.1.1	Long Short-Term Memory (LSTM)	19
4.1.2	Gated Recurrent Units (GRU)	23
5	Attention Mechanism	26
5.1	Bahdanau Attention (Additive Attention)	26
5.2	Luong Attention (Multiplicative Attention)	26
6	Transformer Architecture	32
6.1	Self-Attention Mechanism	32
6.2	Multi-Head Attention	32
6.3	BERT (Bidirectional Encoder Representations from Transformers)	38

7	Advanced NLP Topics	45
7.1	Named Entity Recognition (NER)	45
7.2	Question Answering	49
7.3	Text Summarization	53
8	Advanced Language Models	60
8.1	GPT (Generative Pre-trained Transformer)	60
9	Emerging Trends and Future Directions	66
9.1	Large Language Models (LLMs)	66
9.2	Multimodal Models	66
9.3	Retrieval-Augmented Generation (RAG)	68
9.4	Ethical Considerations in NLP	73
10	Practical NLP Applications	73
10.1	Building a Complete NLP Pipeline	73
11	Conclusion and Best Practices	80
11.1	NLP Project Lifecycle	80
11.2	Model Selection Guide	81
11.3	Performance Optimization	81
11.4	Ethical Guidelines for NLP	86
11.5	Future Directions	86
11.6	Resources for Further Learning	87
12	Final Thoughts	87

1 Introduction to Natural Language Processing

1.1 What is NLP?

Definition 1.1 (Natural Language Processing). *NLP is a field of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language. It sits at the intersection of computer science, linguistics, and machine learning.*

The Challenge: Human language is complex, ambiguous, and context-dependent. Unlike programming languages, natural languages have:

- Ambiguity (words/phrases with multiple meanings)
- Context-dependence (meaning changes with context)
- Idioms and metaphors
- Cultural references
- Sarcasm and irony

1.2 Applications of NLP

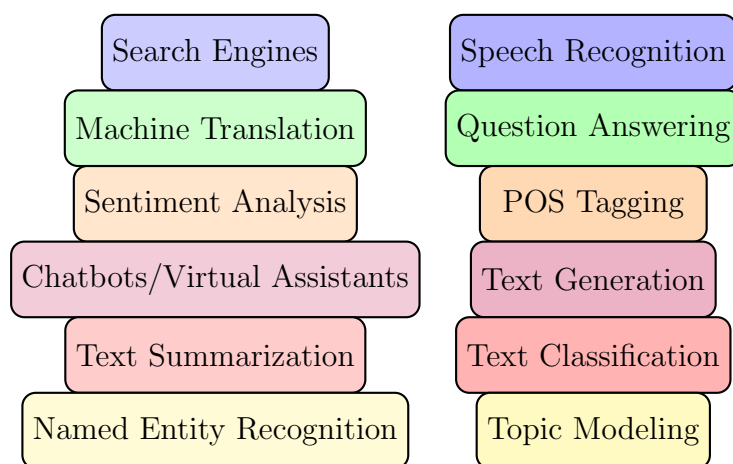


Figure 1: Applications of Natural Language Processing

1.3 The NLP Pipeline

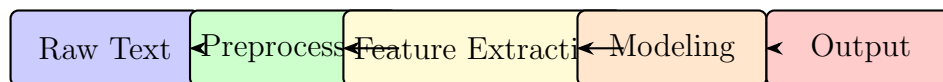


Figure 2: The NLP Processing Pipeline

2 Text Preprocessing

2.1 Tokenization

What is Tokenization? The process of splitting text into smaller units called tokens (words, subwords, or characters).

```
1 import nltk
2 import spacy
3 from transformers import AutoTokenizer
4
5 # Sample text
6 text = "Dr. Smith moved to the U.S.A. in 2020. He said: 'Hello world!'"
7
8 # 1. Word Tokenization with NLTK
9 nltk.download('punkt')
10 from nltk.tokenize import word_tokenize, sent_tokenize
11
12 print("NLTK Word Tokenization:")
13 print(word_tokenize(text))
14 print("\nNLTK Sentence Tokenization:")
15 print(sent_tokenize(text))
16
17 # 2. Advanced tokenization with spaCy
18 nlp = spacy.load("en_core_web_sm")
19 doc = nlp(text)
20 print("\nspacey Tokenization:")
21 print([token.text for token in doc])
22
23 # 3. Subword tokenization with BERT
24 tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
25 print("\nBERT Tokenization:")
26 print(tokenizer.tokenize(text))
27
28 # 4. Custom tokenization rules
29 from nltk.tokenize import RegexpTokenizer
30
31 # Keep words and numbers
32 tokenizer = RegexpTokenizer(r'\w+')
33 print("\nCustom Regex Tokenization:")
34 print(tokenizer.tokenize(text))
```

Listing 1: Tokenization Examples

2.2 Text Normalization

Definition 2.1 (Text Normalization). *Transforming text into a canonical form to reduce variability while preserving meaning.*

Common Techniques:

1. **Lowercasing:** Convert all text to lowercase

2. **Stemming:** Reduce words to their root form (aggressive)
3. **Lemmatization:** Reduce words to dictionary form (linguistically accurate)
4. **Removing Noise:** URLs, HTML tags, special characters
5. **Expanding Contractions:** "don't" → "do not"
6. **Correcting Spelling:** Fix common spelling errors

```
1 import nltk
2 from nltk.stem import PorterStemmer, WordNetLemmatizer
3 import re
4 from autocorrect import Speller
5
6 nltk.download('wordnet')
7 nltk.download('omw-eng')
8
9 text = "The cats are running quickly. Don't you think they're amazing?"
10
11 # 1. Lowercasing
12 lower_text = text.lower()
13 print("Lowercasing:", lower_text)
14
15 # 2. Stemming
16 stemmer = PorterStemmer()
17 tokens = nltk.word_tokenize(text)
18 stemmed = [stemmer.stem(token) for token in tokens]
19 print("\nStemming:", ' '.join(stemmed))
20
21 # 3. Lemmatization
22 lemmatizer = WordNetLemmatizer()
23 lemmatized = [lemmatizer.lemmatize(token, pos='v') for token in tokens] #
24 print("\nLemmatization:", ' '.join(lemmatized))
25
26 # 4. Remove special characters and numbers
27 clean_text = re.sub(r'[^\w\s]', '', text)
28 print("\nRemoved special chars:", clean_text)
29
30 # 5. Expand contractions (simplified)
31 contractions = {
32     "don't": "do not",
33     "doesn't": "does not",
34     "isn't": "is not",
35     "they're": "they are"
36 }
37
38 for contraction, expansion in contractions.items():
39     text = text.replace(contraction, expansion)
40 print("\nExpanded contractions:", text)
41
42 # 6. Spell correction
43 spell = Speller(lang='en')
```

```

44 misspelled = "I luv natural langage procesing"
45 corrected = spell(misspelled)
46 print("\nSpell correction:", corrected)

```

Listing 2: Text Normalization Techniques

2.3 Stop Words Removal

Stop Words: Common words that carry little semantic meaning (the, is, at, which, etc.)

```

1 from nltk.corpus import stopwords
2 from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS
3
4 nltk.download('stopwords')
5
6 text = "The quick brown fox jumps over the lazy dog"
7 tokens = nltk.word_tokenize(text.lower())
8
9 # NLTK stop words
10 stop_words_nltk = set(stopwords.words('english'))
11 filtered_nltk = [word for word in tokens if word not in stop_words_nltk]
12 print("NLTK stop words removed:", filtered_nltk)
13
14 # Scikit-learn stop words
15 filtered_sklearn = [word for word in tokens if word not in
16                     ENGLISH_STOP_WORDS]
17 print("Scikit-learn stop words removed:", filtered_sklearn)
18
19 # Custom stop words
20 custom_stop_words = {'quick', 'brown', 'lazy'}
21 filtered_custom = [word for word in tokens if word not in
22                   custom_stop_words]
23 print("Custom stop words removed:", filtered_custom)
24
25 # When NOT to remove stop words?
26 # - Question Answering: "to be or not to be" - all words important
27 # - Sentiment Analysis: "not good" - "not" is crucial
28 # - Phrase detection: "New York" - both words needed

```

Listing 3: Stop Words Removal

2.4 Part-of-Speech Tagging

Definition 2.2 (Part-of-Speech Tagging). *Assigning grammatical categories (noun, verb, adjective, etc.) to each word in a sentence.*

POS Tag	Description	Example
NN	Noun, singular	cat, house
NNS	Noun, plural	cats, houses
VB	Verb, base form	run, eat
VBD	Verb, past tense	ran, ate
JJ	Adjective	quick, beautiful
RB	Adverb	quickly, very
IN	Preposition	in, on, at
DT	Determiner	the, a, an
PRP	Pronoun	I, you, he
CC	Coordinating conjunction	and, but, or

Table 1: Common POS tags in the Penn Treebank tagset

```

1 import nltk
2 import spacy
3
4 nltk.download('averaged_perceptron_tagger')
5 nlp = spacy.load("en_core_web_sm")
6
7 sentences = [
8     "The quick brown fox jumps over the lazy dog.",
9     "I love natural language processing!",
10    "She is reading a book about machine learning."
11 ]
12
13 print("NLTK POS Tagging:")
14 for sentence in sentences:
15     tokens = nltk.word_tokenize(sentence)
16     tags = nltk.pos_tag(tokens)
17     print(f"\n{sentence}")
18     for token, tag in tags:
19         print(f" {token:15} -> {tag}")
20
21 print("\n" + "="*60)
22 print("spaCy POS Tagging (with dependencies):")
23
24 for sentence in sentences:
25     doc = nlp(sentence)
26     print(f"\n{sentence}")
27     for token in doc:
28         print(f" {token.text:15} -> {token.pos_:10} {token.dep_:15} {
29             token.head.text}")
30
31 # Visualize dependencies
32 from spacy import displacy
33 doc = nlp("The quick brown fox jumps over the lazy dog.")
34 displacy.render(doc, style="dep", jupyter=True)
35
36 # Using POS tags for specific tasks

```

```

36 def extract_noun_phrases(sentence):
37     """Extract noun phrases using POS patterns."""
38     doc = nlp(sentence)
39     noun_phrases = []
40     current_phrase = []
41
42     for token in doc:
43         if token.pos_ in ['NOUN', 'PROPN', 'ADJ', 'DET']:
44             current_phrase.append(token.text)
45         elif current_phrase:
46             noun_phrases.append(' '.join(current_phrase))
47             current_phrase = []
48
49     if current_phrase:
50         noun_phrases.append(' '.join(current_phrase))
51
52     return noun_phrases
53
54 print("\nNoun phrases extraction:")
55 for sentence in sentences:
56     phrases = extract_noun_phrases(sentence)
57     print(f"{sentence} -> {phrases}")

```

Listing 4: POS Tagging Implementation

3 Text Representation

The Fundamental Problem: Computers understand numbers, not text. We need to convert text into numerical representations.

3.1 One-Hot Encoding

Definition 3.1 (One-Hot Encoding). *Represent each word as a binary vector where only one position is 1 (hot) and all others are 0.*

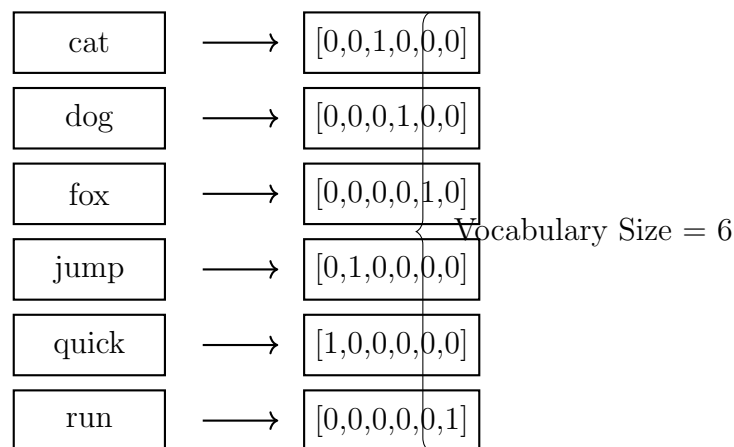


Figure 3: One-hot encoding creates sparse vectors

Limitations:

- **Dimensionality:** Vocabulary size can be huge (100k+ words)
- **Sparsity:** Most entries are zeros
- **No semantic relationships:** All words are equally different

3.2 Bag of Words (BoW)

Definition 3.2 (Bag of Words). *Represent text as a vector of word counts, ignoring word order and grammar.*

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 import pandas as pd
3
4 documents = [
5     "I love natural language processing",
6     "Natural language processing is amazing",
7     "I love machine learning and natural language processing",
8     "Machine learning is a subset of artificial intelligence"
9 ]
10
11 # Create CountVectorizer
12 vectorizer = CountVectorizer()
13
14 # Fit and transform
15 X = vectorizer.fit_transform(documents)
16
17 # Convert to DataFrame for visualization
18 df_bow = pd.DataFrame(
19     X.toarray(),
20     columns=vectorizer.get_feature_names_out()
21 )
22
23 print("Bag of Words Representation:")
24 print(df_bow)
25
26 # With n-grams (capturing some word order)
27 bigram_vectorizer = CountVectorizer(ngram_range=(1, 2))
28 X_bigram = bigram_vectorizer.fit_transform(documents)
29 df_bigram = pd.DataFrame(
30     X_bigram.toarray(),
31     columns=bigram_vectorizer.get_feature_names_out()
32 )
33
34 print("\n\nBag of Words with Bigrams (1-2 grams):")
35 print(df_bigram[['love', 'natural language', 'language processing', '
36     machine learning']])
37
38 # TF-IDF variant
39 from sklearn.feature_extraction.text import TfidfVectorizer
```

```

40 tfidf_vectorizer = TfidfVectorizer()
41 X_tfidf = tfidf_vectorizer.fit_transform(documents)
42 df_tfidf = pd.DataFrame(
43     X_tfidf.toarray(),
44     columns=tfidf_vectorizer.get_feature_names_out()
45 )
46
47 print("\n\nTF-IDF Representation:")
48 print(df_tfidf)

```

Listing 5: Bag of Words Implementation

3.3 Term Frequency-Inverse Document Frequency (TF-IDF)

Definition 3.3 (TF-IDF). *A numerical statistic that reflects how important a word is to a document in a collection.*

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total words in } d}$$

$$\text{IDF}(t, D) = \log \frac{\text{total documents}}{\text{documents containing } t}$$

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

```

1 import numpy as np
2 from sklearn.feature_extraction.text import TfidfVectorizer
3
4 corpus = [
5     "the cat sat on the mat",
6     "the dog sat on the log",
7     "cats and dogs are pets",
8     "pets are amazing companions"
9 ]
10
11 # Manual TF-IDF calculation for understanding
12 def manual_tf_idf(corpus):
13     documents = [doc.split() for doc in corpus]
14     vocab = sorted(set(word for doc in documents for word in doc))
15
16     # Calculate TF
17     tf = np.zeros((len(documents), len(vocab)))
18     for i, doc in enumerate(documents):
19         for word in doc:
20             tf[i, vocab.index(word)] += 1
21         tf[i] = tf[i] / len(doc) # Normalize
22
23     # Calculate IDF
24     doc_freq = np.zeros(len(vocab))
25     for word in vocab:
26         for doc in documents:
27             if word in doc:

```

```

28         doc_freq[vocab.index(word)] += 1
29
30     idf = np.log(len(documents) / (doc_freq + 1e-6))
31
32     # Calculate TF-IDF
33     tf_idf = tf * idf
34
35     return tf_idf, vocab
36
37 tf_idf_manual, vocab = manual_tf_idf(corpus)
38 print("Manual TF-IDF calculation:")
39 for i, doc in enumerate(corpus):
40     print(f"\nDocument {i+1}: {doc}")
41     for j, word in enumerate(vocab):
42         if tf_idf_manual[i, j] > 0:
43             print(f"    {word}: {tf_idf_manual[i, j]:.3f}")
44
45 # Using sklearn
46 vectorizer = TfidfVectorizer()
47 X = vectorizer.fit_transform(corpus)
48
49 print("\n\nSklearn TF-IDF:")
50 feature_names = vectorizer.get_feature_names_out()
51 for i, doc in enumerate(corpus):
52     print(f"\nDocument {i+1}: {doc}")
53     for j, word in enumerate(feature_names):
54         value = X[i, j]
55         if value > 0:
56             print(f"    {word}: {value:.3f}")
57
58 # Document similarity using TF-IDF
59 from sklearn.metrics.pairwise import cosine_similarity
60
61 similarity_matrix = cosine_similarity(X)
62 print("\n\nDocument Similarity Matrix:")
63 print(similarity_matrix)
64
65 # Interpretation
66 print("\n\nInsights from TF-IDF:")
67 print("1. Common words (the, on) get low IDF scores")
68 print("2. Unique words (cats, dogs, amazing) get high IDF scores")
69 print("3. Documents with similar word distributions have high cosine
    similarity")

```

Listing 6: TF-IDF Implementation and Analysis

3.4 Word Embeddings

The Breakthrough: Instead of sparse vectors, learn dense vectors that capture semantic meaning.

Word Embeddings: Words with Similar Meaning are Close in Vector Space

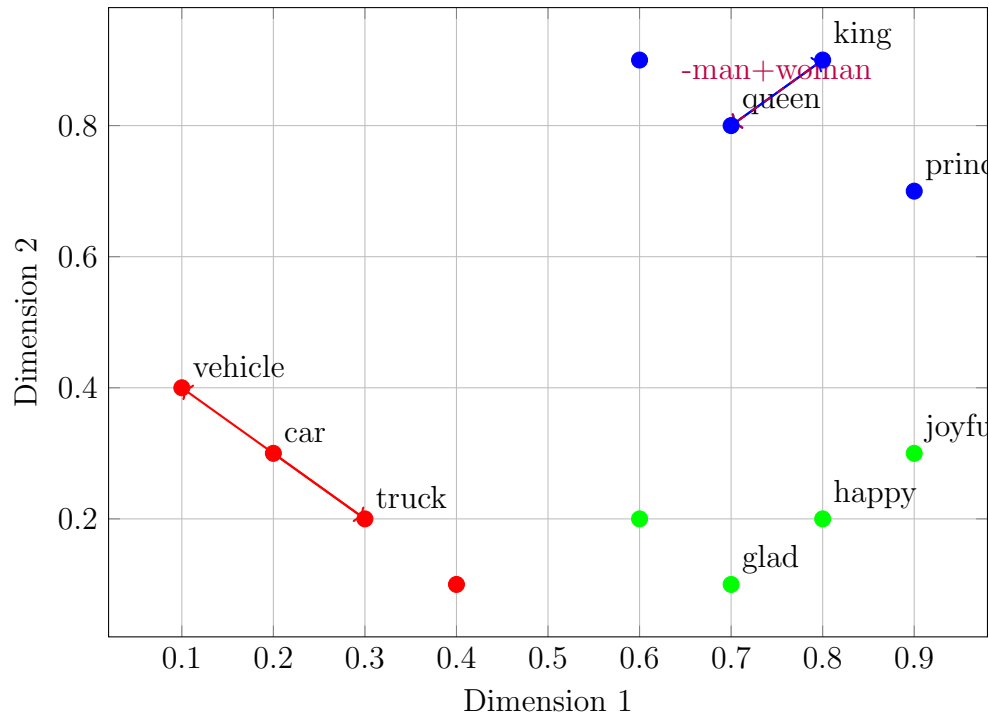


Figure 4: Word embeddings capture semantic relationships in vector space

3.4.1 Word2Vec

Definition 3.4 (Word2Vec). A neural network model that learns word embeddings by predicting words from their context (CBOW) or context from words (Skip-gram).

CBOW: Predict target word from context **Skip-gram:** Predict context from target word

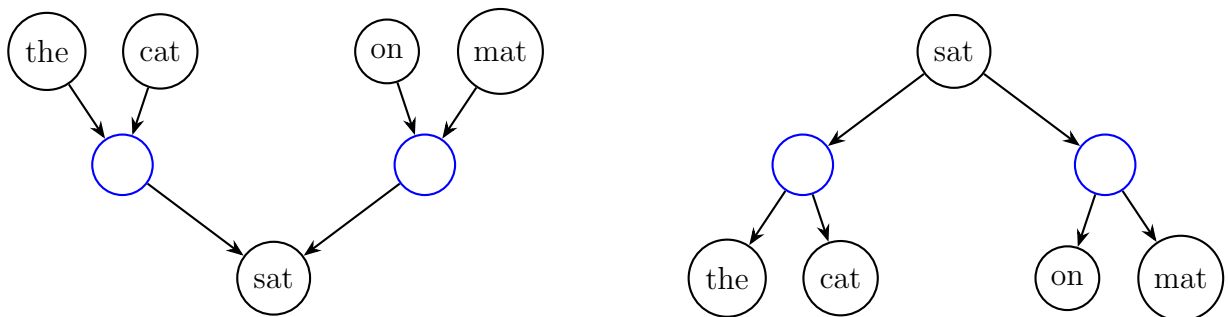


Figure 5: Word2Vec architectures: CBOW and Skip-gram

```
1 import gensim
2 from gensim.models import Word2Vec
```

```

3 import nltk
4 from nltk.corpus import brown
5 import numpy as np
6
7 nltk.download('brown')
8
9 # Load sample corpus
10 sentences = brown.sents()[:1000] # First 1000 sentences
11
12 print(f"Number of sentences: {len(sentences)}")
13 print(f"Sample sentence: {sentences[0]}")
14
15 # Train Word2Vec model
16 model = Word2Vec(
17     sentences=sentences,
18     vector_size=100, # Dimension of word vectors
19     window=5, # Context window size
20     min_count=2, # Ignore words with freq < 2
21     workers=4, # Number of CPU cores
22     sg=1 # 1 for skip-gram, 0 for CBOW
23 )
24
25 print(f"\nVocabulary size: {len(model.wv)}")
26
27 # Get word vector
28 vector_cat = model.wv['cat']
29 print(f"\nVector for 'cat' (first 10 dimensions): {vector_cat[:10]}")
30 print(f"Vector shape: {vector_cat.shape}")
31
32 # Find similar words
33 similar_words = model.wv.most_similar('man', topn=5)
34 print("\nWords similar to 'man':")
35 for word, score in similar_words:
36     print(f" {word}: {score:.3f}")
37
38 # Word analogy: king - man + woman = ?
39 result = model.wv.most_similar(positive=['woman', 'king'], negative=['man'], topn=3)
40 print("\nking - man + woman = ?")
41 for word, score in result:
42     print(f" {word}: {score:.3f}")
43
44 # Visualize embeddings (2D projection)
45 from sklearn.decomposition import PCA
46 import matplotlib.pyplot as plt
47
48 # Get vectors for some words
49 words = ['king', 'queen', 'man', 'woman', 'prince', 'princess',
50          'car', 'truck', 'vehicle', 'bicycle', 'happy', 'sad', 'angry']
51
52 # Get vectors
53 word_vectors = np.array([model.wv[word] for word in words])
54
55 # Reduce to 2D for visualization

```

```

56 pca = PCA(n_components=2)
57 vectors_2d = pca.fit_transform(word_vectors)
58
59 # Plot
60 plt.figure(figsize=(10, 8))
61 plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1])
62
63 # Add labels
64 for i, word in enumerate(words):
65     plt.annotate(word, xy=(vectors_2d[i, 0], vectors_2d[i, 1]),
66                   xytext=(5, 2), textcoords='offset points')
67
68 plt.title('Word Embeddings Visualization (PCA)')
69 plt.xlabel('PC1')
70 plt.ylabel('PC2')
71 plt.grid(True, alpha=0.3)
72 plt.show()
73
74 # Save and load model
75 model.save("word2vec.model")
76 loaded_model = Word2Vec.load("word2vec.model")
77
78 # Using pre-trained embeddings
79 import gensim.downloader as api
80
81 print("\nLoading pre-trained Google News vectors (takes a minute)...")
82 # Uncomment to download (300 dimensions, 3GB)
83 # wv = api.load('word2vec-google-news-300')
84 # print("Similar to 'computer':", wv.most_similar('computer', topn=3))

```

Listing 7: Word2Vec Implementation with Gensim

3.4.2 GloVe (Global Vectors for Word Representation)

Definition 3.5 (GloVe). *Creates word embeddings by factorizing the word co-occurrence matrix, capturing global statistical information.*

$$J = \sum_{i,j=1}^V f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

where:

- X_{ij} : Number of times word j appears in context of word i
- w_i : Word vector for word i
- $\tilde{w}_j$: Context vector for word j
- f : Weighting function to reduce impact of rare co-occurrences

```

1 from gensim.models import KeyedVectors
2 import numpy as np
3
4 # Load pre-trained GloVe vectors (download from https://nlp.stanford.edu/
  projects/glove/)
5 # glove_file = "glove.6B.100d.txt" # 400K words, 100 dimensions
6
7 # For demonstration, we'll create a simple version
8 def create_cooccurrence_matrix(corpus, window_size=2):
9     """Create word co-occurrence matrix."""
10    vocab = set()
11    for sentence in corpus:
12        vocab.update(sentence)
13    vocab = sorted(vocab)
14    word2idx = {word: i for i, word in enumerate(vocab)}
15
16    # Initialize co-occurrence matrix
17    n = len(vocab)
18    cooccurrence = np.zeros((n, n))
19
20    # Count co-occurrences
21    for sentence in corpus:
22        for i, word in enumerate(sentence):
23            center_idx = word2idx[word]
24            # Look at context window
25            start = max(0, i - window_size)
26            end = min(len(sentence), i + window_size + 1)
27            for j in range(start, end):
28                if i != j: # Don't count the word itself
29                    context_idx = word2idx[sentence[j]]
30                    cooccurrence[center_idx, context_idx] += 1
31
32    return cooccurrence, vocab, word2idx
33
34 # Example corpus
35 corpus = [
36     ["the", "cat", "sat", "on", "the", "mat"],
37     ["the", "dog", "played", "with", "the", "ball"],
38     ["cats", "and", "dogs", "are", "pets"],
39     ["pets", "are", "good", "companions"]
40 ]
41
42 cooccurrence, vocab, word2idx = create_cooccurrence_matrix(corpus)
43 print("Vocabulary:", vocab)
44 print("\nCo-occurrence matrix (first 5x5):")
45 print(cooccurrence[:5, :5])
46
47 # Simple GloVe-like embedding using matrix factorization
48 from sklearn.decomposition import TruncatedSVD
49
50 # Factorize the log co-occurrence matrix
51 log_cooccurrence = np.log(cooccurrence + 1) # Add 1 to avoid log(0)
52 svd = TruncatedSVD(n_components=10) # 10-dimensional embeddings

```

```

53 embeddings = svd.fit_transform(log_cooccurrence)
54
55 print(f"\nWord embeddings shape: {embeddings.shape}")
56 print("\nEmbeddings for 'cat' and 'dog':")
57 print(f"cat: {embeddings[word2idx['cat']][:5]}...")
58 print(f"dog: {embeddings[word2idx['dog']][:5]}...")
59
60 # Similarity calculation
61 def cosine_similarity(vec1, vec2):
62     return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(
        vec2))
63
64 cat_vec = embeddings[word2idx['cat']]
65 dog_vec = embeddings[word2idx['dog']]
66 the_vec = embeddings[word2idx['the']]
67
68 print(f"\nCosine similarities:")
69 print(f"cat-dog: {cosine_similarity(cat_vec, dog_vec):.3f}")
70 print(f"cat-the: {cosine_similarity(cat_vec, the_vec):.3f}")
71 print(f"dog-the: {cosine_similarity(dog_vec, the_vec):.3f}")
72
73 # Load actual pre-trained GloVe vectors
74 try:
75     # You need to download glove.6B.zip and extract it first
76     glove_path = "glove.6B.100d.txt"
77     glove_model = KeyedVectors.load_word2vec_format(glove_path, binary=
        False, no_header=True)
78
79     print("\nUsing actual GloVe vectors:")
80     print(f"Vector for 'king': {glove_model['king'][:10]}...")
81
82     # Famous analogy
83     result = glove_model.most_similar(positive=['woman', 'king'], negative
        =['man'], topn=3)
84     print("\nking - man + woman = ?")
85     for word, score in result:
86         print(f" {word}: {score:.3f}")
87
88 except FileNotFoundError:
89     print("\nGloVe file not found. Please download from:")
90     print("https://nlp.stanford.edu/projects/glove/")

```

Listing 8: GloVe Implementation

3.4.3 FastText

Definition 3.6 (FastText). *Extension of Word2Vec that represents words as bags of character n -grams, enabling handling of out-of-vocabulary words and morphological similarities.*

Key Innovation: Word = sum of its character n -gram embeddings

$$\text{vector}(\text{"hello"}) = \text{vector}(\text{"h"}) + \text{vector}(\text{"he"}) + \text{vector}(\text{"el"}) + \dots + \text{vector}(\text{"o"})$$

```

1 from gensim.models import FastText
2 import nltk
3 from nltk.corpus import brown
4
5 nltk.download('brown')
6
7 # Sample corpus
8 sentences = brown.sents()[ :1000]
9
10 # Train FastText model
11 model = FastText(
12     sentences=sentences,
13     vector_size=100,      # Dimension of word vectors
14     window=5,            # Context window
15     min_count=2,         # Ignore rare words
16     workers=4,           # Use 4 CPU cores
17     min_n=3,             # Min length of char n-grams
18     max_n=6              # Max length of char n-grams
19 )
20
21 print(f"Vocabulary size: {len(model.wv)}")
22
23 # Get word vector
24 vector = model.wv['processing']
25 print(f"\nVector for 'processing' (first 10 dims): {vector[:10]}")
26
27 # Handle out-of-vocabulary words through subwords
28 print("\nTesting OOV (Out-Of-Vocabulary) handling:")
29 oov_word = "unseenword"
30 if oov_word in model.wv:
31     print(f"'{oov_word}' is in vocabulary")
32 else:
33     # FastText can still give vector using subwords
34     vector_oov = model.wv[oov_word]
35     print(f"Vector for OOV word '{oov_word}': {vector_oov[:10]}...")
36
37 # Find similar words using subword information
38 print("\nWords similar to 'running':")
39 similar = model.wv.most_similar('running', topn=5)
40 for word, score in similar:
41     print(f"    {word}: {score:.3f}")
42
43 # Get all n-grams for a word
44 word = "processing"
45 print(f"\nCharacter n-grams for '{word}':")
46 # FastText generates n-grams internally, but we can simulate:
47 n_grams = []
48 for n in range(3, 7): # min_n=3, max_n=6
49     for i in range(len(word) - n + 1):
50         n_grams.append(word[i:i+n])
51 print(f"    {n_grams}")
52
53 # Compare with Word2Vec

```

```

54 print("\n" + "="*60)
55 print("Comparing FastText and Word2Vec:")
56
57 # Words with similar morphology
58 test_words = ['run', 'running', 'runner', 'runs', 'ran']
59
60 print("\nSimilarity matrix (FastText):")
61 for w1 in test_words:
62     similarities = []
63     for w2 in test_words:
64         if w1 in model.wv and w2 in model.wv:
65             sim = model.wv.similarity(w1, w2)
66             similarities.append(f"{sim:.3f}")
67         else:
68             similarities.append("N/A")
69     print(f"{w1:10} {' '.join(similarities)}")
70
71 # Visualize word relationships
72 import matplotlib.pyplot as plt
73 from sklearn.decomposition import PCA
74
75 words = ['cat', 'cats', 'dog', 'dogs', 'run', 'running', 'jump', 'jumping']
76 vectors = []
77
78 for word in words:
79     if word in model.wv:
80         vectors.append(model.wv[word])
81     else:
82         print(f"Warning: '{word}' not in vocabulary")
83
84 if vectors:
85     vectors = np.array(vectors)
86     pca = PCA(n_components=2)
87     vectors_2d = pca.fit_transform(vectors)
88
89     plt.figure(figsize=(10, 8))
90     plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1])
91
92     for i, word in enumerate(words):
93         if word in model.wv:
94             plt.annotate(word, xy=(vectors_2d[i, 0], vectors_2d[i, 1]),
95                          xytext=(5, 2), textcoords='offset points')
96
97     plt.title('FastText Embeddings (Morphological Similarities)')
98     plt.xlabel('PC1')
99     plt.ylabel('PC2')
100    plt.grid(True, alpha=0.3)
101    plt.show()
102
103 # Load pre-trained FastText vectors
104 try:
105     import fasttext.util
106     # Download English model (will download ~7GB file)

```

```

107 # fasttext.util.download_model('en', if_exists='ignore')
108 # ft = fasttext.load_model('cc.en.300.bin')
109
110 # print("\nPre-trained FastText model loaded")
111 # print(f"Vector for 'neural': {ft.get_word_vector('neural')
112 #       [:10]}...")
113
114 except ImportError:
115     print("\nInstall fasttext library: pip install fasttext")

```

Listing 9: FastText Implementation

4 Sequence Models for NLP

4.1 Recurrent Neural Networks (RNNs)

The Idea: Process sequences by maintaining a hidden state that captures information about previous elements.

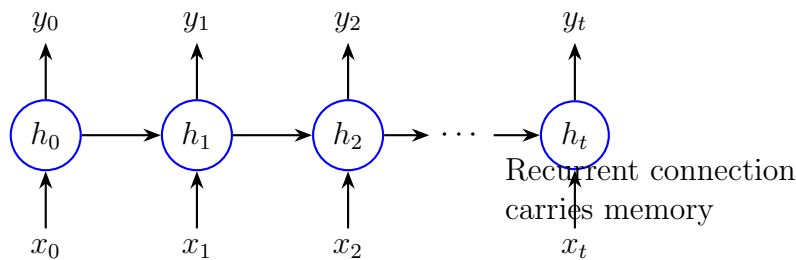


Figure 6: RNN unrolled through time: each step receives input and previous hidden state

RNN Equations:

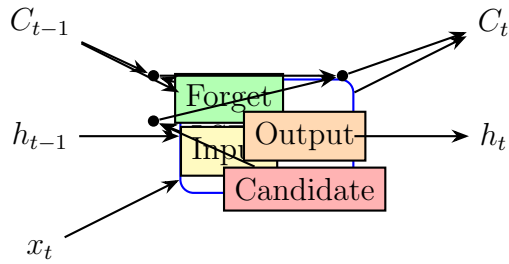
$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

4.1.1 Long Short-Term Memory (LSTM)

The Problem: Vanishing gradients in standard RNNs make it hard to learn long-term dependencies.

Definition 4.1 (LSTM). *A special kind of RNN with gates that control information flow, enabling learning of long-range dependencies.*



Gates:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget})$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input})$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output})$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{Candidate})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$h_t = o_t \odot \tanh(C_t)$$

Figure 7: LSTM cell with gates controlling information flow

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import numpy as np
5 from sklearn.preprocessing import LabelEncoder
6
7 # Sample data: Sentiment analysis
8 sentences = [
9     "I love this movie",
10    "This film is terrible",
11    "Great acting and plot",
12    "Worst movie ever",
13    "Amazing cinematography",
14    "Boring and predictable",
15    "Highly recommended",
16    "Don't waste your time"
17 ]
18 labels = [1, 0, 1, 0, 1, 0, 1, 0] # 1=positive, 0=negative
19
20 # Create vocabulary
21 def build_vocab(sentences):
22     vocab = {'<PAD>': 0, '<UNK>': 1}
23     idx = 2
24     for sentence in sentences:
25         for word in sentence.lower().split():
26             if word not in vocab:
27                 vocab[word] = idx
28                 idx += 1
29     return vocab
30
31 vocab = build_vocab(sentences)
32 print(f"Vocabulary size: {len(vocab)}")
33
34 # Convert sentences to sequences
35 def sentence_to_sequence(sentence, vocab, max_len=10):
36     tokens = sentence.lower().split()
37     sequence = [vocab.get(token, vocab['<UNK>']) for token in tokens]
38     # Pad or truncate

```

```

39     if len(sequence) < max_len:
40         sequence += [vocab['<PAD>']] * (max_len - len(sequence))
41     else:
42         sequence = sequence[:max_len]
43     return sequence
44
45 # Prepare data
46 X = torch.tensor([sentence_to_sequence(s, vocab) for s in sentences],
47                  dtype=torch.long)
48 y = torch.tensor(labels, dtype=torch.float32).unsqueeze(1)
49 print(f"X shape: {X.shape}, y shape: {y.shape}")
50
51 # Define LSTM model
52 class SentimentLSTM(nn.Module):
53     def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
54         super().__init__()
55
56         # Embedding layer
57         self.embedding = nn.Embedding(vocab_size, embedding_dim,
58 padding_idx=0)
59
60         # LSTM layer
61         self.lstm = nn.LSTM(
62             input_size=embedding_dim,
63             hidden_size=hidden_dim,
64             num_layers=1,
65             batch_first=True,
66             bidirectional=False
67         )
68
69         # Output layer
70         self.fc = nn.Linear(hidden_dim, output_dim)
71         self.sigmoid = nn.Sigmoid()
72
73     def forward(self, text):
74         # text shape: [batch_size, seq_length]
75
76         # 1. Embedding layer
77         embedded = self.embedding(text) # [batch_size, seq_length,
78 embedding_dim]
79
80         # 2. LSTM layer
81         lstm_out, (hidden, cell) = self.lstm(embedded)
82         # lstm_out: [batch_size, seq_length, hidden_dim]
83         # hidden: [1, batch_size, hidden_dim]
84
85         # 3. Take the last hidden state
86         last_hidden = hidden[-1] # [batch_size, hidden_dim]
87
88         # 4. Fully connected layer
89         output = self.fc(last_hidden)
90
91         # 5. Sigmoid activation for binary classification

```

```

90         return self.sigmoid(output)
91
92 # Initialize model
93 vocab_size = len(vocab)
94 embedding_dim = 50
95 hidden_dim = 64
96 output_dim = 1
97
98 model = SentimentLSTM(vocab_size, embedding_dim, hidden_dim, output_dim)
99 print(model)
100
101 # Loss and optimizer
102 criterion = nn.BCELoss()
103 optimizer = optim.Adam(model.parameters(), lr=0.001)
104
105 # Training loop
106 num_epochs = 100
107 for epoch in range(num_epochs):
108     model.train()
109
110     # Forward pass
111     predictions = model(X)
112     loss = criterion(predictions, y)
113
114     # Backward pass
115     optimizer.zero_grad()
116     loss.backward()
117
118     # Gradient clipping to prevent exploding gradients
119     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1)
120
121     optimizer.step()
122
123     if (epoch + 1) % 20 == 0:
124         # Calculate accuracy
125         with torch.no_grad():
126             preds = (predictions > 0.5).float()
127             accuracy = (preds == y).float().mean()
128
129             print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {loss.item():.4f},
130               Accuracy: {accuracy.item():.4f}')
131
132 # Test on new sentences
133 test_sentences = [
134     "This was an excellent film",
135     "I didn't like the story",
136     "The acting was superb"
137 ]
138
139 model.eval()
140 with torch.no_grad():
141     for sentence in test_sentences:
142         sequence = sentence_to_sequence(sentence, vocab)
143         tensor = torch.tensor([sequence], dtype=torch.long)

```

```

143     prediction = model(tensor)
144
145     sentiment = "positive" if prediction > 0.5 else "negative"
146     confidence = prediction.item() if prediction > 0.5 else 1 -
prediction.item()
147
148     print(f''{sentence}' -> {sentiment} (confidence: {confidence:.2%})
")
149
150 # Visualize LSTM outputs
151 print("\nLSTM Internals:")
152 sample_input = X[0:1] # First sentence
153 embedded = model.embedding(sample_input)
154 lstm_out, (hidden, cell) = model.lstm(embedded)
155
156 print(f"Input shape: {sample_input.shape}")
157 print(f"Embedded shape: {embedded.shape}")
158 print(f"LSTM output shape: {lstm_out.shape}")
159 print(f"Hidden state shape: {hidden.shape}")
160 print(f"Cell state shape: {cell.shape}")
161
162 print("\nHidden states for each time step (first 5 dimensions):")
163 for i in range(lstm_out.shape[1]):
164     print(f"Step {i}: {lstm_out[0, i, :5].detach().numpy()}")

```

Listing 10: LSTM Implementation with PyTorch

4.1.2 Gated Recurrent Units (GRU)

Definition 4.2 (GRU). *A simplified version of LSTM with only two gates (reset and update), making it computationally more efficient.*

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\
 \tilde{h}_t &= \tanh(W \cdot [r_t \odot h_{t-1}, x_t]) \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t
 \end{aligned}$$

```

1 import torch
2 import torch.nn as nn
3 import time
4
5 # Compare GRU and LSTM on same task
6 class GRUModel(nn.Module):
7     def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim):
8         super().__init__()
9         self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=0)
10        self.gru = nn.GRU(embedding_dim, hidden_dim, batch_first=True)
11        self.fc = nn.Linear(hidden_dim, output_dim)

```

```

12         self.sigmoid = nn.Sigmoid()
13
14     def forward(self, text):
15         embedded = self.embedding(text)
16         gru_out, hidden = self.gru(embedded)
17         output = self.fc(hidden[-1])
18         return self.sigmoid(output)
19
20 # Training comparison
21 def train_model(model, X, y, epochs=100):
22     criterion = nn.BCELoss()
23     optimizer = optim.Adam(model.parameters(), lr=0.001)
24
25     start_time = time.time()
26
27     for epoch in range(epochs):
28         model.train()
29         predictions = model(X)
30         loss = criterion(predictions, y)
31
32         optimizer.zero_grad()
33         loss.backward()
34         optimizer.step()
35
36     training_time = time.time() - start_time
37
38     # Final accuracy
39     with torch.no_grad():
40         preds = (predictions > 0.5).float()
41         accuracy = (preds == y).float().mean()
42
43     return accuracy.item(), training_time
44
45 # Initialize models
46 vocab_size = len(vocab)
47 embedding_dim = 50
48 hidden_dim = 64
49 output_dim = 1
50
51 lstm_model = SentimentLSTM(vocab_size, embedding_dim, hidden_dim,
52                             output_dim)
53 gru_model = GRUModel(vocab_size, embedding_dim, hidden_dim, output_dim)
54
55 print("Model comparison:")
56 print("-" * 50)
57
58 # Count parameters
59 def count_parameters(model):
60     return sum(p.numel() for p in model.parameters() if p.requires_grad)
61
62 print(f"LSTM parameters: {count_parameters(lstm_model):,}")
63 print(f"GRU parameters: {count_parameters(gru_model):,}")
64 print(f"GRU has {count_parameters(lstm_model) - count_parameters(gru_model):,} fewer parameters")

```

```

64
65 # Train and compare
66 lstm_acc, lstm_time = train_model(lstm_model, X, y, epochs=50)
67 gru_acc, gru_time = train_model(gru_model, X, y, epochs=50)
68
69 print(f"\nPerformance comparison:")
70 print(f"LSTM - Accuracy: {lstm_acc:.4f}, Time: {lstm_time:.2f}s")
71 print(f"GRU - Accuracy: {gru_acc:.4f}, Time: {gru_time:.2f}s")
72 print(f"GRU is {lstm_time/gru_time:.2f}x faster")
73
74 # Memory usage comparison
75 print("\nMemory usage for one forward pass:")
76 batch_size = 8
77 seq_length = 10
78
79 # Create sample input
80 sample_input = torch.randint(0, vocab_size, (batch_size, seq_length))
81
82 # Measure memory
83 import torch.cuda as cuda # For GPU memory, if available
84
85 lstm_model.eval()
86 gru_model.eval()
87
88 with torch.no_grad():
89     # LSTM forward pass
90     lstm_output = lstm_model(sample_input)
91
92     # GRU forward pass
93     gru_output = gru_model(sample_input)
94
95 print("Both models produce same shape outputs:")
96 print(f"LSTM output shape: {lstm_output.shape}")
97 print(f"GRU output shape: {gru_output.shape}")
98
99 # When to use which?
100 print("\n" + "="*60)
101 print("When to use LSTM vs GRU:")
102 print("="*60)
103 print("""
104 Use LSTM when:
105     - You need maximum performance on complex sequence tasks
106     - You have long sequences with complex dependencies
107     - You're working with very large datasets
108     - Computational resources are not a constraint
109
110 Use GRU when:
111     - You need faster training and inference
112     - You have limited computational resources
113     - You're working with smaller datasets
114     - You want a simpler model that's easier to train
115     - Your sequences are not extremely long
116
117 In practice:

```

```

118 - For most tasks, GRU works almost as well as LSTM
119 - Try both and see which works better for your specific task
120 - Modern architectures (like Transformers) often outperform both
121 """ )

```

Listing 11: GRU vs LSTM Comparison

5 Attention Mechanism

The Key Innovation: Instead of compressing all information into a fixed-size hidden state, let the model learn where to "pay attention" in the input sequence.

5.1 Bahdanau Attention (Additive Attention)

Definition 5.1 (Bahdanau Attention). *The first attention mechanism proposed for neural machine translation, using a small neural network to compute attention scores.*

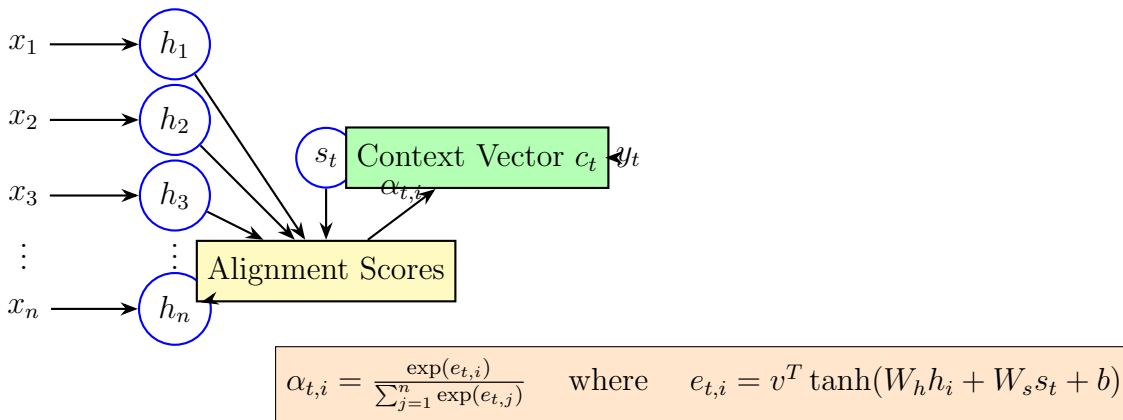


Figure 8: Bahdanau attention: computes alignment scores between decoder state and all encoder states

5.2 Luong Attention (Multiplicative Attention)

Definition 5.2 (Luong Attention). *A simpler attention mechanism that uses dot products instead of a neural network to compute attention scores.*

$$\text{score}(h_t, \bar{h}_s) = \begin{cases} h_t^T \bar{h}_s & \text{dot} \\ h_t^T W_a \bar{h}_s & \text{general} \\ v_a^T \tanh(W_a [h_t; \bar{h}_s]) & \text{concat} \end{cases}$$

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F

```

```

4 import numpy as np
5
6 class BahdanauAttention(nn.Module):
7     """Additive attention (Bahdanau style)"""
8     def __init__(self, hidden_dim):
9         super().__init__()
10        self.W_h = nn.Linear(hidden_dim, hidden_dim, bias=False)
11        self.W_s = nn.Linear(hidden_dim, hidden_dim, bias=False)
12        self.v = nn.Linear(hidden_dim, 1, bias=False)
13
14        def forward(self, decoder_hidden, encoder_outputs):
15            """
16            decoder_hidden: [batch_size, hidden_dim] - current decoder state
17            encoder_outputs: [batch_size, seq_len, hidden_dim] - all encoder
18            states
19            """
20            # Add time dimension to decoder hidden state
21            decoder_hidden = decoder_hidden.unsqueeze(1) # [batch_size, 1,
22            hidden_dim]
23
24            # Calculate energy scores
25            # encoder_outputs: [batch_size, seq_len, hidden_dim]
26            # decoder_hidden: [batch_size, 1, hidden_dim]
27
28            # Transform encoder outputs
29            encoder_features = self.W_h(encoder_outputs) # [batch_size,
30            seq_len, hidden_dim]
31
32            # Transform decoder hidden state
33            decoder_features = self.W_s(decoder_hidden) # [batch_size, 1,
34            hidden_dim]
35
36            # Calculate energy (additive)
37            # Expand decoder features to match encoder sequence length
38            decoder_features = decoder_features.expand_as(encoder_features)
39
40            # Combine
41            energy = torch.tanh(encoder_features + decoder_features) # [
42            batch_size, seq_len, hidden_dim]
43
44            # Calculate attention scores
45            attention_scores = self.v(energy).squeeze(-1) # [batch_size,
46            seq_len]
47            attention_weights = F.softmax(attention_scores, dim=1)
48
49            # Calculate context vector
50            # attention_weights: [batch_size, seq_len]
51            # encoder_outputs: [batch_size, seq_len, hidden_dim]
52            context = torch.bmm(attention_weights.unsqueeze(1),
53            encoder_outputs) # [batch_size, 1, hidden_dim]
54            context = context.squeeze(1) # [batch_size, hidden_dim]
55
56            return context, attention_weights

```

```

51 class LuongAttention(nn.Module):
52     """Multiplicative attention (Luong style)"""
53     def __init__(self, hidden_dim, method='general'):
54         super().__init__()
55         self.method = method
56
57         if method == 'general':
58             self.W = nn.Linear(hidden_dim, hidden_dim, bias=False)
59         elif method == 'concat':
60             self.W = nn.Linear(hidden_dim * 2, hidden_dim, bias=False)
61             self.v = nn.Linear(hidden_dim, 1, bias=False)
62
63     def forward(self, decoder_hidden, encoder_outputs):
64         """
65         decoder_hidden: [batch_size, hidden_dim] - current decoder state
66         encoder_outputs: [batch_size, seq_len, hidden_dim] - all encoder
67         states
68         """
69         batch_size, seq_len, hidden_dim = encoder_outputs.size()
70
71         if self.method == 'dot':
72             # decoder_hidden: [batch_size, hidden_dim]
73             # encoder_outputs: [batch_size, seq_len, hidden_dim]
74             decoder_hidden = decoder_hidden.unsqueeze(2) # [batch_size,
75             hidden_dim, 1]
76             energy = torch.bmm(encoder_outputs, decoder_hidden) # [
77             batch_size, seq_len, 1]
78             energy = energy.squeeze(2) # [batch_size, seq_len]
79
80         elif self.method == 'general':
81             # Transform encoder outputs
82             encoder_transformed = self.W(encoder_outputs) # [batch_size,
83             seq_len, hidden_dim]
84
85             # decoder_hidden: [batch_size, hidden_dim]
86             decoder_hidden = decoder_hidden.unsqueeze(2) # [batch_size,
87             hidden_dim, 1]
88             energy = torch.bmm(encoder_transformed, decoder_hidden) # [
89             batch_size, seq_len, 1]
90             energy = energy.squeeze(2) # [batch_size, seq_len]
91
92         elif self.method == 'concat':
93             # Expand decoder hidden to match sequence length
94             decoder_hidden = decoder_hidden.unsqueeze(1).expand(batch_size
95             , seq_len, hidden_dim)
96
97             # Concatenate
98             concat = torch.cat((decoder_hidden, encoder_outputs), dim=2)
99             # [batch_size, seq_len, hidden_dim*2]
100             energy = self.W(concat) # [batch_size, seq_len, hidden_dim]
101             energy = self.v(torch.tanh(energy)).squeeze(2) # [batch_size,
102             seq_len]
103
104             # Calculate attention weights

```

```

96         attention_weights = F.softmax(energy, dim=1)
97
98         # Calculate context vector
99         context = torch.bmm(attention_weights.unsqueeze(1),
100 encoder_outputs) # [batch_size, 1, hidden_dim]
101         context = context.squeeze(1) # [batch_size, hidden_dim]
102
103         return context, attention_weights
104
105 # Example: Machine Translation with Attention
106 class Seq2SeqAttention(nn.Module):
107     """Sequence-to-sequence model with attention for machine translation"""
108
109     def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim,
110 dropout=0.5):
111         super().__init__()
112
113         # Encoder
114         self.embedding = nn.Embedding(vocab_size, embedding_dim,
115 padding_idx=0)
116         self.encoder_lstm = nn.LSTM(embedding_dim, hidden_dim,
117                                     batch_first=True, bidirectional=True)
118
119         # Decoder
120         self.decoder_lstm = nn.LSTM(embedding_dim + hidden_dim * 2,
121                                     hidden_dim, batch_first=True)
122
123         # Attention
124         self.attention = BahdanauAttention(hidden_dim * 2)
125
126         # Output layer
127         self.fc_out = nn.Linear(hidden_dim * 3, output_dim)
128         self.dropout = nn.Dropout(dropout)
129
130     def forward(self, src, trg, teacher_forcing_ratio=0.5):
131         """
132         src: [batch_size, src_len] - source language tokens
133         trg: [batch_size, trg_len] - target language tokens
134         """
135         batch_size, src_len = src.shape
136         trg_len = trg.shape[1]
137
138         # Initialize outputs
139         outputs = torch.zeros(batch_size, trg_len, self.fc_out.
140 out_features).to(src.device)
141
142         # ===== ENCODER =====
143         embedded_src = self.dropout(self.embedding(src)) # [batch_size,
144 src_len, embedding_dim]
145
146         # Encoder forward pass
147         encoder_outputs, (hidden, cell) = self.encoder_lstm(embedded_src)
148         # encoder_outputs: [batch_size, src_len, hidden_dim*2] (
149 bidirectional)

```

```

143     # hidden, cell: [2, batch_size, hidden_dim]
144
145     # Use final forward and backward hidden states
146     hidden = torch.cat((hidden[-2,:,:], hidden[-1,:,:]), dim=1).
unsqueeze(0)
147     cell = torch.cat((cell[-2,:,:], cell[-1,:,:]), dim=1).unsqueeze(0)
148
149     # ===== DECODER =====
150     # First input to decoder is <sos> token
151     input = trg[:, 0].unsqueeze(1) # [batch_size, 1]
152
153     for t in range(1, trg_len):
154         # Embed input token
155         embedded_trg = self.dropout(self.embedding(input)) # [
batch_size, 1, embedding_dim]
156
157         # Calculate attention
158         context, attention_weights = self.attention(hidden.squeeze(0),
encoder_outputs)
159         # context: [batch_size, hidden_dim*2]
160         # attention_weights: [batch_size, src_len]
161
162         # Concatenate context with embedded input
163         lstm_input = torch.cat((embedded_trg, context.unsqueeze(1)),
dim=2)
164         # lstm_input: [batch_size, 1, embedding_dim + hidden_dim*2]
165
166         # Decoder LSTM step
167         output, (hidden, cell) = self.decoder_lstm(lstm_input, (hidden
, cell))
168         # output: [batch_size, 1, hidden_dim]
169
170         # Concatenate context with decoder output
171         output = torch.cat((output.squeeze(1), context), dim=1)
172         # output: [batch_size, hidden_dim * 3]
173
174         # Final output layer
175         prediction = self.fc_out(output)
176         outputs[:, t, :] = prediction
177
178         # Decide whether to use teacher forcing
179         teacher_force = np.random.random() < teacher_forcing_ratio
180
181         # Get the highest predicted token
182         top1 = prediction.argmax(1)
183
184         # If teacher forcing, use actual next token as next input
185         # Otherwise, use predicted token
186         input = trg[:, t].unsqueeze(1) if teacher_force else top1.
unsqueeze(1)
187
188     return outputs
189
190 # Test the attention mechanism

```

```

191 print("Testing Attention Mechanisms")
192 print("="*60)
193
194 # Create sample data
195 batch_size = 4
196 seq_len = 10
197 hidden_dim = 128
198
199 # Simulate encoder outputs (like from a bidirectional LSTM)
200 encoder_outputs = torch.randn(batch_size, seq_len, hidden_dim * 2)
201 decoder_hidden = torch.randn(batch_size, hidden_dim * 2)
202
203 print(f"Encoder outputs shape: {encoder_outputs.shape}")
204 print(f"Decoder hidden shape: {decoder_hidden.shape}")
205
206 # Test Bahdanau attention
207 bahdanau_attn = BahdanauAttention(hidden_dim * 2)
208 context_b, weights_b = bahdanau_attn(decoder_hidden, encoder_outputs)
209 print(f"\nBahdanau Attention:")
210 print(f"Context vector shape: {context_b.shape}")
211 print(f"Attention weights shape: {weights_b.shape}")
212 print(f"Attention weights sum to 1: {weights_b.sum(dim=1)}")
213
214 # Test Luong attention (different methods)
215 methods = ['dot', 'general', 'concat']
216 for method in methods:
217     luong_attn = LuongAttention(hidden_dim * 2, method=method)
218     context_l, weights_l = luong_attn(decoder_hidden, encoder_outputs)
219     print(f"\nLuong Attention ({method}):")
220     print(f"Context vector shape: {context_l.shape}")
221     print(f"Attention weights shape: {weights_l.shape}")
222
223 # Visualize attention weights
224 import matplotlib.pyplot as plt
225
226 # Create a simple example
227 src_sentence = ["I", "love", "natural", "language", "processing"]
228 trg_sentence = ["J'", "aime", "le", "traitement", "du", "langage", "naturel"]
229
230 # Simulate attention weights (aligned for example)
231 attention_weights = torch.tensor([
232     [0.8, 0.1, 0.05, 0.03, 0.02], # "J'" attends to "I"
233     [0.1, 0.7, 0.1, 0.05, 0.05], # "aime" attends to "love"
234     [0.05, 0.1, 0.6, 0.2, 0.05], # "le" attends to "natural"
235     [0.02, 0.08, 0.1, 0.7, 0.1], # "traitement" attends to "language"
236     [0.01, 0.04, 0.05, 0.1, 0.8], # "du" attends to "processing"
237     [0.3, 0.2, 0.2, 0.2, 0.1], # "langage" attends to multiple
238     [0.2, 0.2, 0.3, 0.2, 0.1] # "naturel" attends to multiple
239 ])
240
241 plt.figure(figsize=(10, 8))
242 plt.imshow(attention_weights.numpy(), cmap='YlOrRd', aspect='auto')
243 plt.colorbar(label='Attention Weight')

```

```

244 plt.xticks(range(len(src_sentence)), src_sentence, rotation=45)
245 plt.yticks(range(len(trg_sentence)), trg_sentence)
246
247
248 plt.xlabel('Source (English)')
249 plt.ylabel('Target (French)')
250 plt.title('Attention Weights Visualization')
251 plt.tight_layout()
252 plt.show()

```

Listing 12: Attention Mechanism Implementation

6 Transformer Architecture

The Revolution: Transformers replaced RNNs with self-attention, enabling parallel processing and capturing long-range dependencies more effectively.

6.1 Self-Attention Mechanism

Definition 6.1 (Self-Attention). *Allows each position in the sequence to attend to all positions in the same sequence, capturing dependencies regardless of distance.*

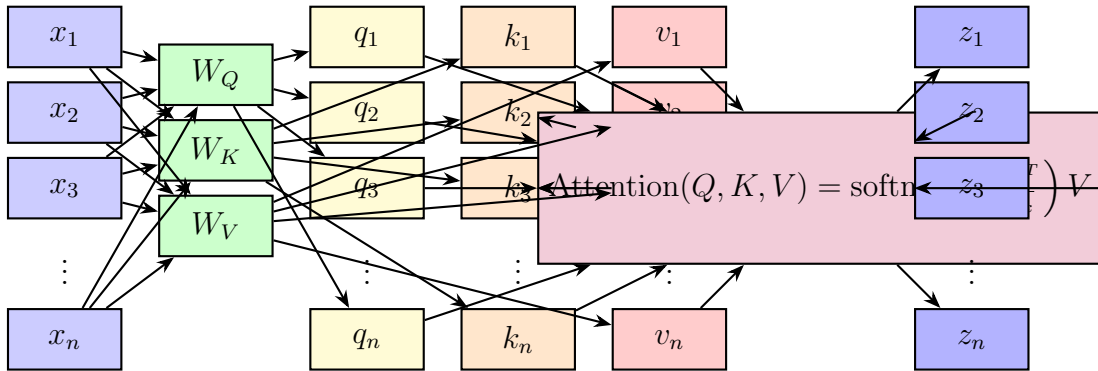


Figure 9: Self-attention mechanism: each position gets a weighted combination of all positions

6.2 Multi-Head Attention

Definition 6.2 (Multi-Head Attention). *Run self-attention multiple times in parallel with different learned linear projections, then concatenate the results.*

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5 import numpy as np
6
7 class MultiHeadAttention(nn.Module):
8     def __init__(self, d_model, num_heads, dropout=0.1):
9         super().__init__()
10         assert d_model % num_heads == 0, "d_model must be divisible by
num_heads"
11
12         self.d_model = d_model
13         self.num_heads = num_heads
14         self.d_k = d_model // num_heads
15
16         # Linear layers for Q, K, V
17         self.W_q = nn.Linear(d_model, d_model)
18         self.W_k = nn.Linear(d_model, d_model)
19         self.W_v = nn.Linear(d_model, d_model)
20
21         # Output projection
22         self.W_o = nn.Linear(d_model, d_model)
23
24         self.dropout = nn.Dropout(dropout)
25
26     def scaled_dot_product_attention(self, Q, K, V, mask=None):
27         """
28         Q: [batch_size, num_heads, seq_len, d_k]
29         K: [batch_size, num_heads, seq_len, d_k]
30         V: [batch_size, num_heads, seq_len, d_k]
31         mask: [batch_size, 1, seq_len, seq_len]
32         """
33         # Calculate attention scores
34         scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k
)
35         # scores: [batch_size, num_heads, seq_len, seq_len]
36
37         # Apply mask (if provided)
38         if mask is not None:
39             scores = scores.masked_fill(mask == 0, -1e9)
40
41         # Apply softmax
42         attention_weights = F.softmax(scores, dim=-1)
43         attention_weights = self.dropout(attention_weights)
44
45         # Apply attention to values
46         output = torch.matmul(attention_weights, V)
47         # output: [batch_size, num_heads, seq_len, d_k]
48
49         return output, attention_weights
50
51     def forward(self, Q, K, V, mask=None):

```

```

52     batch_size, seq_len, d_model = Q.size()
53
54     # Linear projections and split into heads
55     Q = self.W_q(Q).view(batch_size, -1, self.num_heads, self.d_k).
transpose(1, 2)
56     K = self.W_k(K).view(batch_size, -1, self.num_heads, self.d_k).
transpose(1, 2)
57     V = self.W_v(V).view(batch_size, -1, self.num_heads, self.d_k).
transpose(1, 2)
58
59     # Apply attention
60     attention_output, attention_weights = self.
scaled_dot_product_attention(Q, K, V, mask)
61
62     # Concatenate heads
63     attention_output = attention_output.transpose(1, 2).contiguous().
view(batch_size, -1, d_model)
64
65     # Final linear projection
66     output = self.W_o(attention_output)
67
68     return output, attention_weights
69
70 class PositionwiseFeedForward(nn.Module):
71     def __init__(self, d_model, d_ff, dropout=0.1):
72         super().__init__()
73         self.linear1 = nn.Linear(d_model, d_ff)
74         self.linear2 = nn.Linear(d_ff, d_model)
75         self.dropout = nn.Dropout(dropout)
76         self.activation = nn.GELU() # GELU works better than ReLU for
Transformers
77
78     def forward(self, x):
79         return self.linear2(self.dropout(self.activation(self.linear1(x)))
)
80
81 class PositionalEncoding(nn.Module):
82     def __init__(self, d_model, max_len=5000):
83         super().__init__()
84
85         # Create positional encoding matrix
86         pe = torch.zeros(max_len, d_model)
87         position = torch.arange(0, max_len, dtype=torch.float).unsqueeze
(1)
88         div_term = torch.exp(torch.arange(0, d_model, 2).float() *
(-math.log(10000.0) / d_model))
89
90         pe[:, 0::2] = torch.sin(position * div_term)
91         pe[:, 1::2] = torch.cos(position * div_term)
92
93         pe = pe.unsqueeze(0) # Add batch dimension
94         self.register_buffer('pe', pe)
95
96     def forward(self, x):

```

```

98         # x: [batch_size, seq_len, d_model]
99         return x + self.pe[:, :x.size(1)]
100
101 class TransformerEncoderLayer(nn.Module):
102     def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
103         super().__init__()
104
105         # Multi-head attention
106         self.self_attn = MultiHeadAttention(d_model, num_heads, dropout)
107
108         # Feed-forward network
109         self.feed_forward = PositionwiseFeedForward(d_model, d_ff, dropout
110     )
111
112         # Layer normalization
113         self.norm1 = nn.LayerNorm(d_model)
114         self.norm2 = nn.LayerNorm(d_model)
115
116         # Dropout
117         self.dropout = nn.Dropout(dropout)
118
119     def forward(self, x, mask=None):
120         # Self-attention with residual connection
121         attn_output, attn_weights = self.self_attn(x, x, x, mask)
122         x = self.norm1(x + self.dropout(attn_output))
123
124         # Feed-forward with residual connection
125         ff_output = self.feed_forward(x)
126         x = self.norm2(x + self.dropout(ff_output))
127
128         return x, attn_weights
129
130 class TransformerEncoder(nn.Module):
131     def __init__(self, vocab_size, d_model, num_heads, num_layers, d_ff,
132         max_len, dropout=0.1):
133         super().__init__()
134
135         # Token embedding
136         self.embedding = nn.Embedding(vocab_size, d_model)
137
138         # Positional encoding
139         self.positional_encoding = PositionalEncoding(d_model, max_len)
140
141         # Dropout
142         self.dropout = nn.Dropout(dropout)
143
144         # Encoder layers
145         self.layers = nn.ModuleList([
146             TransformerEncoderLayer(d_model, num_heads, d_ff, dropout)
147             for _ in range(num_layers)
148         ])
149
150     def forward(self, x, mask=None):
151         # x: [batch_size, seq_len]

```

```

150
151     # Embed tokens
152     x = self.embedding(x) # [batch_size, seq_len, d_model]
153
154     # Add positional encoding
155     x = self.positional_encoding(x)
156     x = self.dropout(x)
157
158     # Pass through encoder layers
159     all_attention_weights = []
160     for layer in self.layers:
161         x, attn_weights = layer(x, mask)
162         all_attention_weights.append(attn_weights)
163
164     return x, all_attention_weights
165
166 class TransformerForClassification(nn.Module):
167     def __init__(self, vocab_size, d_model, num_heads, num_layers, d_ff,
168                 max_len, num_classes, dropout=0.1):
169         super().__init__()
170
171         # Transformer encoder
172         self.encoder = TransformerEncoder(vocab_size, d_model, num_heads,
173                                         num_layers, d_ff, max_len, dropout)
174
175         # Classification head
176         self.classifier = nn.Sequential(
177             nn.Linear(d_model, d_model // 2),
178             nn.ReLU(),
179             nn.Dropout(dropout),
180             nn.Linear(d_model // 2, num_classes)
181         )
182
183     def forward(self, x, mask=None):
184         # Encode input
185         encoded, attention_weights = self.encoder(x, mask)
186
187         # Use [CLS] token representation for classification
188         # In practice, we'd use the first token
189         cls_representation = encoded[:, 0, :] # [batch_size, d_model]
190
191         # Classify
192         logits = self.classifier(cls_representation)
193
194         return logits, attention_weights
195
196 # Test the transformer
197 print("Testing Transformer Implementation")
198 print("="*60)
199
200 # Hyperparameters
201 vocab_size = 10000
202 d_model = 512
203 num_heads = 8

```

```

202 num_layers = 6
203 d_ff = 2048
204 max_len = 128
205 num_classes = 2
206 batch_size = 4
207 seq_len = 32
208
209 # Create model
210 model = TransformerForClassification(vocab_size, d_model, num_heads,
211                                     num_layers, d_ff, max_len,
212                                     num_classes)
213
214 print(f"Model architecture:")
215 print(f"- Embedding dimension: {d_model}")
216 print(f"- Number of heads: {num_heads}")
217 print(f"- Number of layers: {num_layers}")
218 print(f"- Feed-forward dimension: {d_ff}")
219
220 # Count parameters
221 def count_parameters(model):
222     return sum(p.numel() for p in model.parameters() if p.requires_grad)
223
224 print(f"\nTotal parameters: {count_parameters(model):,}")
225
226 # Create sample input
227 input_tokens = torch.randint(0, vocab_size, (batch_size, seq_len))
228 print(f"\nInput shape: {input_tokens.shape}")
229
230 # Create attention mask (optional)
231 attention_mask = torch.ones(batch_size, 1, 1, seq_len)
232 print(f"Attention mask shape: {attention_mask.shape}")
233
234 # Forward pass
235 logits, attention_weights = model(input_tokens, attention_mask)
236 print(f"\nOutput logits shape: {logits.shape}")
237 print(f"Number of attention weight tensors: {len(attention_weights)}")
238 print(f"Single attention weight shape: {attention_weights[0].shape}")
239
240 # Visualize attention patterns
241 print("\nVisualizing attention patterns...")
242
243 # Take first sample, first head, first layer
244 attention_sample = attention_weights[0][0, 0].detach().numpy()
245
246 import matplotlib.pyplot as plt
247
248 plt.figure(figsize=(10, 8))
249 plt.imshow(attention_sample, cmap='viridis', aspect='auto')
250 plt.colorbar(label='Attention Weight')
251 plt.xlabel('Key Position')
252 plt.ylabel('Query Position')
253 plt.title('Self-Attention Pattern (First Layer, First Head)')
254 plt.tight_layout()
255 plt.show()

```

```

255
256 # Compare with PyTorch's built-in transformer
257 print("\n" + "="*60)
258 print("Comparing with PyTorch's built-in Transformer")
259
260 # PyTorch's implementation
261 pytorch_transformer = nn.TransformerEncoder(
262     nn.TransformerEncoderLayer(d_model=d_model, nhead=num_heads,
263                               dim_feedforward=d_ff, batch_first=True),
264     num_layers=num_layers
265 )
266
267 # Test with same input (need to add embedding first)
268 embedding = nn.Embedding(vocab_size, d_model)
269 embedded_input = embedding(input_tokens)
270 pos_encoding = PositionalEncoding(d_model, max_len)
271 embedded_input = pos_encoding(embedded_input)
272
273 # PyTorch transformer expects [seq_len, batch_size, d_model] by default
274 # but we use batch_first=True
275 pytorch_output = pytorch_transformer(embedded_input)
276 print(f"PyTorch transformer output shape: {pytorch_output.shape}")
277
278 print("\nCustom vs PyTorch implementation:")
279 print("- Both implement multi-head attention")
280 print("- Both have feed-forward networks")
281 print("- Both use residual connections and layer normalization")
282 print("- PyTorch's version is highly optimized")
283 print("- Our implementation is easier to understand and modify")

```

Listing 13: Transformer Implementation from Scratch

6.3 BERT (Bidirectional Encoder Representations from Transformers)

Definition 6.3 (BERT). *A transformer-based model pre-trained on large text corpora using masked language modeling and next sentence prediction objectives.*

Key Innovations:

1. **Bidirectional:** Considers both left and right context simultaneously
2. **Masked Language Model (MLM):** Randomly masks tokens and predicts them
3. **Next Sentence Prediction (NSP):** Predicts if sentence B follows sentence A
4. **Pre-training + Fine-tuning:** Train on large corpus, then fine-tune on specific tasks

```

1 from transformers import BertTokenizer, BertModel,
   BertForSequenceClassification
2 from transformers import AdamW, get_linear_schedule_with_warmup
3 import torch

```

```

4 from torch.utils.data import Dataset, DataLoader
5 import pandas as pd
6 from sklearn.model_selection import train_test_split
7 from sklearn.metrics import accuracy_score, classification_report
8 import numpy as np
9
10 # Load pre-trained BERT model and tokenizer
11 print("Loading BERT model and tokenizer...")
12 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
13 model = BertModel.from_pretrained('bert-base-uncased')
14
15 print(f"Vocabulary size: {tokenizer.vocab_size}")
16 print(f"Model parameters: {sum(p.numel() for p in model.parameters()):,}")
17
18 # Sample text
19 text = "I love natural language processing with transformers!"
20
21 # Tokenize
22 tokens = tokenizer.tokenize(text)
23 token_ids = tokenizer.encode(text, add_special_tokens=True)
24
25 print(f"\nText: {text}")
26 print(f"Tokens: {tokens}")
27 print(f"Token IDs: {token_ids}")
28 print(f"Decoded: {tokenizer.decode(token_ids)}")
29
30 # BERT input format
31 inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=
    True)
32 print(f"\nBERT input keys: {inputs.keys()}")
33 print(f"input_ids shape: {inputs['input_ids'].shape}")
34 print(f"attention_mask shape: {inputs['attention_mask'].shape}")
35 print(f"token_type_ids shape: {inputs.get('token_type_ids', 'Not provided
    ').shape if inputs.get('token_type_ids') is not None else 'Not provided
    '}")
36
37 # Get BERT embeddings
38 with torch.no_grad():
39     outputs = model(**inputs)
40
41 print(f"\nBERT outputs:")
42 print(f"Last hidden state shape: {outputs.last_hidden_state.shape}")
43 print(f"Pooler output shape: {outputs.pooler_output.shape}")
44
45 # Extract specific token embeddings
46 # [CLS] token embedding (used for classification)
47 cls_embedding = outputs.last_hidden_state[0, 0, :]
48 print(f"\n[CLS] token embedding (first 10 dims): {cls_embedding[:10]}")
49
50 # Word embeddings
51 word_embeddings = outputs.last_hidden_state[0, 1:-1, :] # Exclude [CLS]
    and [SEP]
52 print(f"Word embeddings shape: {word_embeddings.shape}")
53

```

```

54 # Example: Sentiment analysis with BERT
55 class SentimentDataset(Dataset):
56     def __init__(self, texts, labels, tokenizer, max_len=128):
57         self.texts = texts
58         self.labels = labels
59         self.tokenizer = tokenizer
60         self.max_len = max_len
61
62     def __len__(self):
63         return len(self.texts)
64
65     def __getitem__(self, idx):
66         text = str(self.texts[idx])
67         label = self.labels[idx]
68
69         encoding = self.tokenizer.encode_plus(
70             text,
71             add_special_tokens=True,
72             max_length=self.max_len,
73             padding='max_length',
74             truncation=True,
75             return_tensors='pt',
76             return_attention_mask=True
77         )
78
79         return {
80             'input_ids': encoding['input_ids'].flatten(),
81             'attention_mask': encoding['attention_mask'].flatten(),
82             'labels': torch.tensor(label, dtype=torch.long)
83         }
84
85 # Create sample dataset
86 data = {
87     'text': [
88         "I love this movie, it's amazing!",
89         "This film is terrible and boring.",
90         "Great acting and wonderful plot.",
91         "Worst movie I've ever seen.",
92         "The cinematography is stunning.",
93         "I fell asleep during the movie.",
94         "Highly recommended for everyone.",
95         "Don't waste your time on this."
96     ],
97     'label': [1, 0, 1, 0, 1, 0, 1, 0] # 1=positive, 0=negative
98 }
99 df = pd.DataFrame(data)
100
101 # Split data
102 train_texts, val_texts, train_labels, val_labels = train_test_split(
103     df['text'], df['label'], test_size=0.25, random_state=42
104 )
105
106 # Create datasets
107 train_dataset = SentimentDataset(train_texts.tolist(), train_labels.tolist()

```

```

    (), tokenizer)
108 val_dataset = SentimentDataset(val_texts.tolist(), val_labels.tolist(),
    tokenizer)
109
110 # Create data loaders
111 train_loader = DataLoader(train_dataset, batch_size=2, shuffle=True)
112 val_loader = DataLoader(val_dataset, batch_size=2)
113
114 # Load BERT for sequence classification
115 classifier = BertForSequenceClassification.from_pretrained(
116     'bert-base-uncased',
117     num_labels=2, # Binary classification
118     output_attentions=False,
119     output_hidden_states=False
120 )
121
122 # Training setup
123 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
124 classifier.to(device)
125
126 optimizer = AdamW(classifier.parameters(), lr=2e-5, eps=1e-8)
127 epochs = 3
128 total_steps = len(train_loader) * epochs
129 scheduler = get_linear_schedule_with_warmup(
130     optimizer,
131     num_warmup_steps=0,
132     num_training_steps=total_steps
133 )
134
135 # Training function
136 def train_epoch(model, data_loader, optimizer, scheduler, device):
137     model.train()
138     losses = []
139     correct_predictions = 0
140
141     for batch in data_loader:
142         input_ids = batch['input_ids'].to(device)
143         attention_mask = batch['attention_mask'].to(device)
144         labels = batch['labels'].to(device)
145
146         optimizer.zero_grad()
147
148         outputs = model(
149             input_ids=input_ids,
150             attention_mask=attention_mask,
151             labels=labels
152         )
153
154         loss = outputs.loss
155         logits = outputs.logits
156
157         losses.append(loss.item())
158
159     # Calculate accuracy

```

```

160     preds = torch.argmax(logits, dim=1)
161     correct_predictions += torch.sum(preds == labels)
162
163     loss.backward()
164     torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
165     optimizer.step()
166     scheduler.step()
167
168     return np.mean(losses), correct_predictions.double() / len(data_loader
169 .dataset)
170
171 # Evaluation function
172 def eval_model(model, data_loader, device):
173     model.eval()
174     losses = []
175     correct_predictions = 0
176
177     with torch.no_grad():
178         for batch in data_loader:
179             input_ids = batch['input_ids'].to(device)
180             attention_mask = batch['attention_mask'].to(device)
181             labels = batch['labels'].to(device)
182
183             outputs = model(
184                 input_ids=input_ids,
185                 attention_mask=attention_mask,
186                 labels=labels
187             )
188
189             loss = outputs.loss
190             logits = outputs.logits
191
192             losses.append(loss.item())
193
194             preds = torch.argmax(logits, dim=1)
195             correct_predictions += torch.sum(preds == labels)
196
197     return np.mean(losses), correct_predictions.double() / len(data_loader
198 .dataset)
199
200 # Training loop
201 print("\n" + "="*60)
202 print("Fine-tuning BERT for sentiment analysis")
203 print("="*60)
204
205 for epoch in range(epochs):
206     print(f'\nEpoch {epoch + 1}/{epochs}')
207     print('-' * 40)
208
209     train_loss, train_acc = train_epoch(
210         classifier, train_loader, optimizer, scheduler, device
211     )
212
213     val_loss, val_acc = eval_model(classifier, val_loader, device)

```

```

212     print(f'Train loss: {train_loss:.4f}, Train accuracy: {train_acc:.4f}',
213     )
214     print(f'Val loss: {val_loss:.4f}, Val accuracy: {val_acc:.4f}')
215
216 # Test on new sentences
217 print("\n" + "="*60)
218 print("Testing fine-tuned BERT model")
219 print("="*60)
220
221 test_sentences = [
222     "This movie was absolutely fantastic!",
223     "I hated every minute of this film.",
224     "The acting was mediocre but the story was good.",
225     "Waste of time and money."
226 ]
227
228 classifier.eval()
229 with torch.no_grad():
230     for sentence in test_sentences:
231         encoding = tokenizer.encode_plus(
232             sentence,
233             add_special_tokens=True,
234             max_length=128,
235             padding='max_length',
236             truncation=True,
237             return_tensors='pt'
238         )
239
240         input_ids = encoding['input_ids'].to(device)
241         attention_mask = encoding['attention_mask'].to(device)
242
243         outputs = classifier(input_ids=input_ids, attention_mask=
attention_mask)
244         logits = outputs.logits
245         probs = torch.softmax(logits, dim=1)
246
247         pred = torch.argmax(probs, dim=1).item()
248         confidence = probs[0][pred].item()
249
250         sentiment = "positive" if pred == 1 else "negative"
251
252         print(f''{sentence}'')
253         print(f"    Sentiment: {sentiment} (confidence: {confidence:.2%})")
254         print(f"    Probabilities: negative={probs[0][0]:.2%}, positive={
probs[0][1]:.2%}")
255         print()
256
257 # BERT attention visualization
258 print("\n" + "="*60)
259 print("Visualizing BERT attention")
260 print("="*60)
261
262 # Load model with attention outputs

```

```

263 model_with_attention = BertModel.from_pretrained(
264     'bert-base-uncased',
265     output_attentions=True
266 )
267
268 text = "The cat sat on the mat."
269 inputs = tokenizer(text, return_tensors="pt")
270
271 with torch.no_grad():
272     outputs = model_with_attention(**inputs)
273     attention = outputs.attentions # Tuple of attention tensors
274
275 print(f"Number of layers: {len(attention)}")
276 print(f"Attention shape per layer: {attention[0].shape}")
277 # Shape: [batch_size, num_heads, seq_len, seq_len]
278
279 # Visualize attention from a specific layer and head
280 import matplotlib.pyplot as plt
281
282 layer = 5 # Middle layer
283 head = 3 # Specific attention head
284
285 attention_map = attention[layer][0, head].numpy()
286 tokens = tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])
287
288 plt.figure(figsize=(10, 8))
289 plt.imshow(attention_map, cmap='viridis', aspect='auto')
290 plt.colorbar(label='Attention Weight')
291
292 plt.xticks(range(len(tokens)), tokens, rotation=45)
293 plt.yticks(range(len(tokens)), tokens)
294
295 plt.xlabel('Key Tokens')
296 plt.ylabel('Query Tokens')
297 plt.title(f'BERT Attention - Layer {layer+1}, Head {head+1}')
298 plt.tight_layout()
299 plt.show()
300
301 # BERT embeddings for semantic similarity
302 print("\n" + "="*60)
303 print("Semantic similarity with BERT embeddings")
304 print("="*60)
305
306 sentences = [
307     "The cat sits on the mat",
308     "A feline is resting on a rug",
309     "Dogs are playing in the park",
310     "The weather is sunny today"
311 ]
312
313 # Get BERT embeddings for each sentence
314 embeddings = []
315 for sentence in sentences:
316     inputs = tokenizer(sentence, return_tensors="pt", padding=True,

```

```

truncation=True)
317     with torch.no_grad():
318         outputs = model(**inputs)
319         # Use [CLS] token embedding
320         cls_embedding = outputs.last_hidden_state[0, 0, :]
321         embeddings.append(cls_embedding.numpy())
322
323 embeddings = np.array(embeddings)
324
325 # Calculate cosine similarities
326 from sklearn.metrics.pairwise import cosine_similarity
327
328 similarity_matrix = cosine_similarity(embeddings)
329
330 print("\nSemantic similarity matrix:")
331 for i in range(len(sentences)):
332     print(f"\n'{sentences[i]}'")
333     for j in range(len(sentences)):
334         if i != j:
335             print(f"    Similarity with '{sentences[j][:30]}...': {
similarity_matrix[i][j]:.3f}")

```

Listing 14: BERT Implementation and Usage

7 Advanced NLP Topics

7.1 Named Entity Recognition (NER)

Definition 7.1 (Named Entity Recognition). *The task of identifying and classifying named entities (people, organizations, locations, etc.) in text.*

```

1 import spacy
2 from spacy import displacy
3 import pandas as pd
4
5 # Load spaCy model with NER
6 nlp = spacy.load("en_core_web_sm")
7
8 # Sample text
9 text = """
10 Apple is looking at buying U.K. startup for $1 billion.
11 Tim Cook, CEO of Apple, said the deal would be completed by 2024.
12 The company is based in Cupertino, California.
13 """
14
15 # Process text
16 doc = nlp(text)
17
18 # Extract named entities
19 print("Named Entities Found:")
20 print("="*60)

```

```

21
22 entities = []
23 for ent in doc.ents:
24     entities.append({
25         "Text": ent.text,
26         "Start": ent.start_char,
27         "End": ent.end_char,
28         "Label": ent.label_,
29         "Description": spacy.explain(ent.label_)
30     })
31
32 df_entities = pd.DataFrame(entities)
33 print(df_entities.to_string(index=False))
34
35 # Visualize NER
36 print("\n" + "="*60)
37 print("NER Visualization:")
38 displacy.render(doc, style="ent", jupyter=True)
39
40 # Custom NER training
41 def train_custom_ner():
42     """Example of training a custom NER model (simplified)"""
43     import random
44     from spacy.training import Example
45
46     # Training data format
47     TRAIN_DATA = [
48         ("Uber blew through $1 million a week", {
49             'entities': [(0, 4, 'ORG')] # Uber is an organization
50         }),
51         ("Google rebranded to Alphabet", {
52             'entities': [(0, 6, 'ORG'), (21, 28, 'ORG')]
53         }),
54         ("I like London and Berlin.", {
55             'entities': [(7, 13, 'LOC'), (18, 24, 'LOC')]
56         })
57     ]
58
59     # Create blank model
60     nlp = spacy.blank('en')
61
62     # Add NER pipeline
63     if 'ner' not in nlp.pipe_names:
64         ner = nlp.add_pipe('ner')
65     else:
66         ner = nlp.get_pipe('ner')
67
68     # Add labels
69     for _, annotations in TRAIN_DATA:
70         for ent in annotations.get('entities'):
71             ner.add_label(ent[2])
72
73     # Training (simplified - in practice need more data and epochs)
74     print("Training custom NER model...")

```

```

75
76 # Disable other pipes during training
77 other_pipes = [pipe for pipe in nlp.pipe_names if pipe != 'ner']
78 with nlp.disable_pipes(*other_pipes):
79     optimizer = nlp.begin_training()
80
81     for epoch in range(10):
82         random.shuffle(TRAIN_DATA)
83         losses = {}
84
85         for text, annotations in TRAIN_DATA:
86             example = Example.from_dict(nlp.make_doc(text),
87 annotations)
88             nlp.update([example], drop=0.35, losses=losses)
89
90             print(f"Epoch {epoch+1}, Loss: {losses['ner']:.2f}")
91
92     return nlp
93
94 # Test on custom domain
95 medical_text = """
96 Patient John Smith, 45, was admitted to Mount Sinai Hospital
97 with symptoms of pneumonia. Dr. Emily Johnson prescribed
98 amoxicillin 500mg three times daily for 7 days.
99 """
100 # Process with medical NER (would need medical-trained model)
101 doc_medical = nlp(medical_text)
102
103 print("\n" + "="*60)
104 print("Medical Text NER (with general model):")
105 for ent in doc_medical.ents:
106     print(f"{ent.text:20} -> {ent.label_:10} ({spacy.explain(ent.label_)})")
107
108 # NER with transformers (BERT)
109 from transformers import AutoTokenizer, AutoModelForTokenClassification
110 from transformers import pipeline
111
112 print("\n" + "="*60)
113 print("NER with BERT (transformers):")
114
115 # Load pre-trained NER model
116 tokenizer = AutoTokenizer.from_pretrained("dslim/bert-base-NER")
117 model = AutoModelForTokenClassification.from_pretrained("dslim/bert-base-
118 NER")
119
120 # Create NER pipeline
121 ner_pipeline = pipeline("ner", model=model, tokenizer=tokenizer,
122 aggregation_strategy="simple")
123
124 # Run NER
125 ner_results = ner_pipeline(text)
126 print("\nBERT NER Results:")

```

```

125 for result in ner_results:
126     print(f"{result['word']:20} -> {result['entity_group']:15} (score: {
        result['score']:.3f})")
127
128 # Compare spaCy vs BERT
129 print("\n" + "="*60)
130 print("Comparison: spaCy vs BERT for NER")
131 print("="*60)
132
133 test_sentence = "Elon Musk is the CEO of Tesla and SpaceX, companies based
        in California."
134
135 # spaCy
136 doc_spacy = nlp(test_sentence)
137 print("spaCy NER:")
138 for ent in doc_spacy.ents:
139     print(f"    {ent.text:15} -> {ent.label_}")
140
141 # BERT
142 bert_results = ner_pipeline(test_sentence)
143 print("\nBERT NER:")
144 for result in bert_results:
145     print(f"    {result['word']:15} -> {result['entity_group']}")
146
147 print("\n" + "="*60)
148 print("NER Evaluation Metrics")
149
150 # Simulated evaluation
151 true_entities = [
152     {"text": "Elon Musk", "label": "PERSON", "start": 0, "end": 9},
153     {"text": "Tesla", "label": "ORG", "start": 23, "end": 28},
154     {"text": "SpaceX", "label": "ORG", "start": 33, "end": 39},
155     {"text": "California", "label": "LOC", "start": 67, "end": 77}
156 ]
157
158 predicted_entities = [
159     {"text": "Elon Musk", "label": "PERSON", "start": 0, "end": 9},
160     {"text": "Tesla", "label": "ORG", "start": 23, "end": 28},
161     {"text": "SpaceX", "label": "ORG", "start": 33, "end": 39}
162     # Missing "California"
163 ]
164
165 def calculate_ner_metrics(true_ents, pred_ents):
166     """Calculate precision, recall, F1 for NER"""
167     # Convert to sets for comparison
168     true_set = set((ent["text"], ent["label"]) for ent in true_ents)
169     pred_set = set((ent["text"], ent["label"]) for ent in pred_ents)
170
171     # True positives
172     tp = len(true_set.intersection(pred_set))
173
174     # Precision
175     precision = tp / len(pred_set) if len(pred_set) > 0 else 0
176

```

```

177     # Recall
178     recall = tp / len(true_set) if len(true_set) > 0 else 0
179
180     # F1 score
181     f1 = 2 * precision * recall / (precision + recall) if (precision +
182         recall) > 0 else 0
183
184     return precision, recall, f1
185
186 precision, recall, f1 = calculate_ner_metrics(true_entities,
187     predicted_entities)
188 print(f"Precision: {precision:.3f}")
189 print(f"Recall: {recall:.3f}")
190 print(f"F1 Score: {f1:.3f}")

```

Listing 15: Named Entity Recognition

7.2 Question Answering

Definition 7.2 (Question Answering). *The task of automatically answering questions posed in natural language by extracting or generating answers from a given context.*

```

1 from transformers import pipeline
2 import torch
3 from collections import OrderedDict
4
5 # Load pre-trained QA model
6 print("Loading Question Answering model...")
7 qa_pipeline = pipeline("question-answering",
8     model="distilbert-base-cased-distilled-squad")
9
10 # Sample context and questions
11 context = """
12 The Transformer is a deep learning model introduced in 2017 by researchers
13 at Google.
14 It is based entirely on attention mechanisms and has become the foundation
15 for many
16 state-of-the-art natural language processing models. Unlike recurrent
17 neural networks,
18 Transformers process entire sequences in parallel, making them more
19 efficient for
20 training on large datasets. The original Transformer architecture consists
21 of an
22 encoder and a decoder, but variations like BERT use only the encoder part.
23 """
24
25 questions = [
26     "What year was the Transformer introduced?",
27     "Who introduced the Transformer?",
28     "What is the Transformer based on?",
29     "How do Transformers differ from RNNs?",
30     "What are the two main parts of the original Transformer?"

```

```

26 ]
27
28 print("\nContext:")
29 print(context)
30
31 print("\n" + "="*60)
32 print("Question Answering Results")
33 print("="*60)
34
35 for i, question in enumerate(questions, 1):
36     result = qa_pipeline(question=question, context=context)
37     print(f"\nQ{i}: {question}")
38     print(f"Answer: {result['answer']}")
39     print(f"Confidence: {result['score']:.3f}")
40     print(f"Start/End: {result['start']}/{result['end']}")
41
42 # Custom QA with multiple contexts
43 print("\n" + "="*60)
44 print("QA with Multiple Documents")
45 print("="*60)
46
47 documents = [
48     """Albert Einstein was a German-born theoretical physicist who
49     developed
50     the theory of relativity, one of the two pillars of modern physics."""
51     ,
52     """Marie Curie was a Polish-born physicist and chemist who conducted
53     pioneering research on radioactivity. She was the first woman to win a
54     Nobel Prize."""
55     ,
56     """Isaac Newton was an English mathematician, physicist, and
57     astronomer
58     who formulated the laws of motion and universal gravitation."""
59 ]
60
61 questions = [
62     "Who developed the theory of relativity?",
63     "Who was the first woman to win a Nobel Prize?",
64     "Who formulated the laws of motion?"
65 ]
66
67 for question in questions:
68     print(f"\nQuestion: {question}")
69
70     best_answer = None
71     best_score = 0
72
73     for doc in documents:
74         result = qa_pipeline(question=question, context=doc)
75         if result['score'] > best_score:
76             best_score = result['score']
77             best_answer = result['answer']
78             best_context = doc[:100] + "..." # Truncate for display

```

```

76
77     if best_answer:
78         print(f"Answer: {best_answer}")
79         print(f"Confidence: {best_score:.3f}")
80         print(f"From: {best_context}")
81
82 # Extractive vs Generative QA
83 print("\n" + "="*60)
84 print("Extractive vs Generative Question Answering")
85 print("="*60)
86
87 print("""
88 Extractive QA (like BERT):
89 - Extracts answer span directly from context
90 - Cannot answer if information not in context
91 - More factual and reliable
92 - Limited to the provided text
93
94 Generative QA (like T5, GPT):
95 - Generates answer based on context and question
96 - Can synthesize information
97 - Can answer even if exact phrase not in context
98 - May hallucinate or be less factual
99 """)
100
101 # Try generative QA
102 try:
103     from transformers import pipeline as gen_pipeline
104
105     # Load T5 for generative QA
106     generator = gen_pipeline("text2text-generation",
107                             model="google/flan-t5-small")
108
109     question = "What is the capital of France?"
110     context = "France is a country in Europe. Its capital city has many
111 museums."
112
113     # Format for T5
114     input_text = f"question: {question} context: {context}"
115
116     result = generator(input_text, max_length=50)
117     print(f"\nGenerative QA with T5:")
118     print(f"Question: {question}")
119     print(f"Context: {context}")
120     print(f"Generated answer: {result[0]['generated_text']}")
121
122 except ImportError:
123     print("\nInstall transformers for generative QA")
124
125 # QA evaluation metrics
126 print("\n" + "="*60)
127 print("QA Evaluation Metrics")
128 print("="*60)

```

```

129 def evaluate_qa(true_answers, predicted_answers):
130     """Simple QA evaluation"""
131     exact_matches = 0
132     f1_scores = []
133
134     for true, pred in zip(true_answers, predicted_answers):
135         # Exact match
136         if true.lower() == pred.lower():
137             exact_matches += 1
138
139         # F1 score (token overlap)
140         true_tokens = set(true.lower().split())
141         pred_tokens = set(pred.lower().split())
142
143         if len(true_tokens) == 0 or len(pred_tokens) == 0:
144             f1 = 0
145         else:
146             tp = len(true_tokens.intersection(pred_tokens))
147             precision = tp / len(pred_tokens)
148             recall = tp / len(true_tokens)
149             f1 = 2 * precision * recall / (precision + recall) if (
precision + recall) > 0 else 0
150
151             f1_scores.append(f1)
152
153     em_score = exact_matches / len(true_answers)
154     avg_f1 = sum(f1_scores) / len(f1_scores)
155
156     return em_score, avg_f1
157
158 # Example evaluation
159 true_answers = [
160     "2017",
161     "researchers at Google",
162     "attention mechanisms",
163     "process entire sequences in parallel",
164     "encoder and decoder"
165 ]
166
167 predicted_answers = [
168     "2017",
169     "Google researchers",
170     "attention mechanisms",
171     "they process sequences in parallel",
172     "encoder and a decoder"
173 ]
174
175 em, f1 = evaluate_qa(true_answers, predicted_answers)
176 print(f"Exact Match: {em:.3f}")
177 print(f"F1 Score: {f1:.3f}")
178
179 # Advanced: QA with document retrieval
180 print("\n" + "="*60)
181 print("QA with Document Retrieval (Retriever-Reader)")

```

```

182 print("="*60)
183
184 import numpy as np
185 from sklearn.feature_extraction.text import TfidfVectorizer
186 from sklearn.metrics.pairwise import cosine_similarity
187
188 # Document collection
189 documents = [
190     "The Eiffel Tower is in Paris, France.",
191     "Paris is the capital city of France.",
192     "France is known for its wine and cheese.",
193     "The Louvre Museum is located in Paris.",
194     "French is the official language of France."
195 ]
196
197 questions = [
198     "Where is the Eiffel Tower?",
199     "What is the capital of France?",
200     "What is France known for?"
201 ]
202
203 # Create TF-IDF vectors
204 vectorizer = TfidfVectorizer()
205 doc_vectors = vectorizer.fit_transform(documents)
206
207 print("Document retrieval for QA:")
208 for question in questions:
209     # Convert question to vector
210     question_vector = vectorizer.transform([question])
211
212     # Find most similar document
213     similarities = cosine_similarity(question_vector, doc_vectors)
214     best_doc_idx = np.argmax(similarities[0])
215
216     print(f"\nQuestion: {question}")
217     print(f"Retrieved document: {documents[best_doc_idx]}")
218
219     # Now answer from retrieved document
220     result = qa_pipeline(question=question, context=documents[best_doc_idx])
221     print(f"Answer: {result['answer']} (confidence: {result['score']:.3f})")

```

Listing 16: Question Answering with BERT

7.3 Text Summarization

Definition 7.3 (Text Summarization). *The task of producing a concise and fluent summary while preserving key information and overall meaning.*

```

1 import nltk
2 from nltk.tokenize import sent_tokenize, word_tokenize

```

```

3 from nltk.corpus import stopwords
4 import string
5 from sklearn.feature_extraction.text import TfidfVectorizer
6 import numpy as np
7 from transformers import pipeline
8
9 # Sample long text
10 long_text = """
11 Natural Language Processing (NLP) is a subfield of artificial intelligence
12 that focuses on enabling computers to understand, interpret, and generate
13 human language. The field has evolved significantly over the years, from
14 rule-based systems to statistical methods, and now to deep learning
    approaches.
15
16 One of the key challenges in NLP is dealing with the ambiguity and
    complexity
17 of human language. Words can have multiple meanings, sentences can be
18 structured in various ways, and context plays a crucial role in
    interpretation.
19
20 Recent advances in transformer models, particularly the development of
    BERT,
21 GPT, and their variants, have revolutionized NLP. These models achieve
22 state-of-the-art performance on a wide range of tasks including machine
23 translation, text summarization, question answering, and sentiment
    analysis.
24
25 Despite these advances, many challenges remain in NLP. These include
26 understanding sarcasm and irony, handling low-resource languages, ensuring
27 fairness and reducing bias in language models, and developing models that
28 truly understand language rather than just pattern matching.
29
30 The future of NLP looks promising with ongoing research in areas like
31 multimodal learning (combining text with images and audio), few-shot
    learning,
32 and more interpretable and controllable language models.
33 """
34
35 print("Original Text:")
36 print("="*80)
37 print(long_text)
38 print("="*80)
39
40 # 1. Extractive Summarization (traditional approach)
41 print("\n" + "="*80)
42 print("1. EXTRACTIVE SUMMARIZATION (Traditional TF-IDF based)")
43 print("="*80)
44
45 def extractive_summary(text, num_sentences=3):
46     """Extract top sentences based on TF-IDF scores"""
47     # Tokenize into sentences
48     sentences = sent_tokenize(text)
49
50     # Remove stopwords and punctuation

```

```

51 stop_words = set(stopwords.words('english'))
52
53 def preprocess(sentence):
54     words = word_tokenize(sentence.lower())
55     return ' '.join([word for word in words
56                       if word not in stop_words and word not in string.
punctuation])
57
58 # Preprocess sentences
59 preprocessed_sentences = [preprocess(sent) for sent in sentences]
60
61 # Calculate TF-IDF
62 vectorizer = TfidfVectorizer()
63 tfidf_matrix = vectorizer.fit_transform(preprocessed_sentences)
64
65 # Calculate sentence scores (sum of TF-IDF scores for words in
sentence)
66 sentence_scores = np.array(tfidf_matrix.sum(axis=1)).flatten()
67
68 # Get top N sentences
69 top_indices = sentence_scores.argsort()[-num_sentences:][::-1]
70 top_indices.sort() # Keep original order
71
72 # Return summary
73 summary_sentences = [sentences[i] for i in top_indices]
74 return ' '.join(summary_sentences)
75
76 extractive_summary_result = extractive_summary(long_text, num_sentences=3)
77 print("Extractive Summary:")
78 print(extractive_summary_result)
79
80 # 2. Abstractive Summarization with Transformers
81 print("\n" + "="*80)
82 print("2. ABSTRACTIVE SUMMARIZATION (Transformers)")
83 print("="*80)
84
85 # Load summarization pipeline
86 summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
87
88 # Generate summary
89 summary_result = summarizer(long_text,
90                             max_length=150,
91                             min_length=50,
92                             do_sample=False)
93
94 abstractive_summary = summary_result[0]['summary_text']
95 print("Abstractive Summary (BART):")
96 print(abstractive_summary)
97
98 # 3. Comparison of different models
99 print("\n" + "="*80)
100 print("3. COMPARISON OF DIFFERENT SUMMARIZATION MODELS")
101 print("="*80)
102

```

```

103 models = {
104     "T5-small": "t5-small",
105     "PEGASUS": "google/pegasus-cnn_dailymail",
106     "BART": "facebook/bart-large-cnn"
107 }
108
109 for model_name, model_id in models.items():
110     try:
111         print(f"\n{model_name}:")
112         summarizer_model = pipeline("summarization", model=model_id)
113
114         # Truncate text if needed (some models have input length limits)
115         truncated_text = long_text[:1000] if len(long_text) > 1000 else
long_text
116
117         result = summarizer_model(truncated_text,
118                                   max_length=100,
119                                   min_length=30,
120                                   do_sample=False)
121
122         print(result[0]['summary_text'][:200] + "...")
123
124     except Exception as e:
125         print(f"    Error with {model_name}: {str(e)}")
126
127 # 4. Evaluation metrics for summarization
128 print("\n" + "="*80)
129 print("4. SUMMARIZATION EVALUATION METRICS")
130 print("="*80)
131
132 # ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
133 print("""
134 ROUGE Metrics:
135 - ROUGE-N: Overlap of N-grams between summary and reference
136 - ROUGE-L: Longest Common Subsequence
137 - ROUGE-W: Weighted LCS
138 - ROUGE-S: Skip-bigram co-occurrence
139
140 ROUGE measures recall: How much of the reference summary is captured?
141 """)
142
143 # Install rouge for evaluation
144 try:
145     from rouge import Rouge
146
147     # Reference summary (human-written)
148     reference_summary = """
149     NLP enables computers to understand human language. It has evolved
150     from
151     rule-based to deep learning approaches. Transformers like BERT and GPT
152     revolutionized the field, though challenges like ambiguity and bias
153     remain.
154     Future research focuses on multimodal learning and interpretability.
155     """

```

```

154
155     # Calculate ROUGE scores
156     rouge = Rouge()
157
158     scores = rouge.get_scores(abstractive_summary, reference_summary)
159
160     print("ROUGE Scores for Abstractive Summary:")
161     for key in scores[0]:
162         print(f"{key}:")
163         for metric, value in scores[0][key].items():
164             print(f"    {metric}: {value:.3f}")
165
166 except ImportError:
167     print("Install rouge: pip install rouge")
168
169 # 5. Custom summarization with different techniques
170 print("\n" + "="*80)
171 print("5. CUSTOM SUMMARIZATION TECHNIQUES")
172 print("="*80)
173
174 class TextSummarizer:
175     def __init__(self):
176         self.stop_words = set(stopwords.words('english'))
177
178     def frequency_based_summary(self, text, num_sentences=3):
179         """Summarize based on word frequency"""
180         sentences = sent_tokenize(text)
181
182         # Calculate word frequencies (excluding stopwords)
183         words = word_tokenize(text.lower())
184         words = [word for word in words
185                 if word not in self.stop_words and word not in string.
186                 punctuation]
187
188         word_freq = nltk.FreqDist(words)
189
190         # Score sentences
191         sentence_scores = {}
192         for i, sentence in enumerate(sentences):
193             sentence_words = word_tokenize(sentence.lower())
194             score = sum([word_freq[word] for word in sentence_words
195                         if word in word_freq])
196             sentence_scores[i] = score
197
198         # Get top sentences
199         top_indices = sorted(sentence_scores,
200                             key=sentence_scores.get,
201                             reverse=True)[:num_sentences]
202         top_indices.sort()
203
204         return ' '.join([sentences[i] for i in top_indices])
205
206     def text_rank_summary(self, text, num_sentences=3):
207         """Simplified TextRank algorithm"""

```

```

207     sentences = sent_tokenize(text)
208
209     # Create similarity matrix
210     n = len(sentences)
211     similarity_matrix = np.zeros((n, n))
212
213     # Preprocess sentences
214     def preprocess(sent):
215         words = word_tokenize(sent.lower())
216         return [word for word in words
217                 if word not in self.stop_words and word not in string.
218                 punctuation]
219
220     processed_sentences = [preprocess(sent) for sent in sentences]
221
222     # Calculate similarities
223     for i in range(n):
224         for j in range(n):
225             if i != j:
226                 set_i = set(processed_sentences[i])
227                 set_j = set(processed_sentences[j])
228
229                 if len(set_i.union(set_j)) > 0:
230                     similarity = len(set_i.intersection(set_j)) / len(
231                         set_i.union(set_j))
232                     similarity_matrix[i][j] = similarity
233
234     # Power iteration for PageRank
235     scores = np.ones(n) / n
236     damping = 0.85
237
238     for _ in range(20):
239         new_scores = np.ones(n) * (1 - damping) / n
240         for i in range(n):
241             for j in range(n):
242                 if similarity_matrix[j][i] > 0:
243                     new_scores[i] += damping * scores[j] *
244                     similarity_matrix[j][i]
245             scores = new_scores
246
247     # Get top sentences
248     top_indices = scores.argsort()[-num_sentences:][::-1]
249     top_indices.sort()
250
251     return ' '.join([sentences[i] for i in top_indices])
252
253 # Test custom summarizers
254 summarizer = TextSummarizer()
255
256 print("Frequency-based summary:")
257 print(summarizer.frequency_based_summary(long_text, num_sentences=3))
258
259 print("\nTextRank summary:")
260 print(summarizer.text_rank_summary(long_text, num_sentences=3))

```

```

258
259 # 6. Domain-specific summarization
260 print("\n" + "="*80)
261 print("6. DOMAIN-SPECIFIC SUMMARIZATION")
262 print("="*80)
263
264 # Scientific paper abstract
265 scientific_text = """
266 In this paper, we propose a novel approach to neural architecture search
267 (NAS) that significantly reduces computational requirements while
268 maintaining
269 competitive performance. Our method, called EfficientNAS, uses weight
270 sharing
271 across child models and a progressive search strategy. We demonstrate that
272 EfficientNAS achieves state-of-the-art results on ImageNet with only 0.3
273 GPU days of search time, compared to previous methods requiring thousands
274 of
275 GPU days. Additionally, we provide theoretical analysis showing the
276 convergence
277 properties of our approach and discuss implications for future NAS
278 research.
279 """
280
281 print("Scientific paper abstract summarization:")
282 science_summary = summarizer(scientific_text,
283                               max_length=100,
284                               min_length=30,
285                               do_sample=False)[0]['summary_text']
286 print(science_summary)
287
288 # News article
289 news_text = """
290 The stock market experienced significant volatility today following the
291 Federal Reserve's announcement of interest rate hikes. The Dow Jones
292 Industrial Average fell 500 points in morning trading before recovering
293 some losses in the afternoon. Technology stocks were particularly hard hit
294 ,
295 with the NASDAQ composite dropping 3%. Analysts attribute the market
296 movements to concerns about inflation and economic growth. The Fed
297 indicated
298 that further rate increases are likely in coming months as part of efforts
299 to control inflation, which remains above target levels.
300 """
301
302 print("\nNews article summarization:")
303 news_summary = summarizer(news_text,
304                             max_length=80,
305                             min_length=30,
306                             do_sample=False)[0]['summary_text']
307 print(news_summary)

```

Listing 17: Text Summarization Techniques

8 Advanced Language Models

8.1 GPT (Generative Pre-trained Transformer)

Definition 8.1 (GPT). *A transformer-based language model that uses only the decoder part of the transformer, trained on large text corpora using autoregressive language modeling.*

Key Features:

- Autoregressive: Predicts next token given previous tokens
- Unidirectional: Only left-to-right context (unlike BERT's bidirectional)
- Generative: Can generate coherent, long-form text
- Few-shot learning: Can perform tasks with just a few examples

```
1 from transformers import AutoTokenizer, AutoModelForCausalLM
2 import torch
3
4 # Load GPT-2 (smaller version of GPT-3)
5 print("Loading GPT-2 model...")
6 tokenizer = AutoTokenizer.from_pretrained("gpt2")
7 model = AutoModelForCausalLM.from_pretrained("gpt2")
8
9 # Set pad token
10 tokenizer.pad_token = tokenizer.eos_token
11
12 print(f"Vocabulary size: {tokenizer.vocab_size}")
13 print(f"Model parameters: {sum(p.numel() for p in model.parameters()):,}")
14
15 # Text generation
16 def generate_text(prompt, max_length=100, temperature=0.7, top_p=0.9):
17     """Generate text from prompt using GPT-2"""
18     # Encode the prompt
19     inputs = tokenizer.encode(prompt, return_tensors="pt")
20
21     # Generate text
22     with torch.no_grad():
23         outputs = model.generate(
24             inputs,
25             max_length=max_length,
26             temperature=temperature,
27             top_p=top_p,
28             do_sample=True,
29             pad_token_id=tokenizer.pad_token_id,
30             eos_token_id=tokenizer.eos_token_id,
31             no_repeat_ngram_size=2
32         )
33
34     # Decode the generated text
35     generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```

36         return generated_text
37
38
39 # Test text generation
40 prompts = [
41     "Once upon a time in a land far away,",
42     "The future of artificial intelligence",
43     "In a world where machines can think,",
44     "The secret to happiness is"
45 ]
46
47 print("\n" + "="*60)
48 print("GPT-2 Text Generation Examples")
49 print("="*60)
50
51 for prompt in prompts:
52     print(f"\nPrompt: {prompt}")
53     generated = generate_text(prompt, max_length=100)
54     print(f"Generated: {generated}")
55     print("-" * 80)
56
57 # Different sampling strategies
58 print("\n" + "="*60)
59 print("Different Sampling Strategies")
60 print("="*60)
61
62 prompt = "The meaning of life is"
63
64 print("\n1. Greedy Decoding (deterministic):")
65 inputs = tokenizer.encode(prompt, return_tensors="pt")
66 with torch.no_grad():
67     greedy_output = model.generate(
68         inputs,
69         max_length=50,
70         do_sample=False,
71         num_beams=1
72     )
73 print(tokenizer.decode(greedy_output[0], skip_special_tokens=True))
74
75 print("\n2. Beam Search (beam=3):")
76 with torch.no_grad():
77     beam_output = model.generate(
78         inputs,
79         max_length=50,
80         num_beams=3,
81         early_stopping=True,
82         no_repeat_ngram_size=2
83     )
84 print(tokenizer.decode(beam_output[0], skip_special_tokens=True))
85
86 print("\n3. Top-k Sampling (k=50):")
87 with torch.no_grad():
88     topk_output = model.generate(
89         inputs,

```

```

90         max_length=50,
91         do_sample=True,
92         top_k=50,
93         temperature=0.7
94     )
95     print(tokenizer.decode(topk_output[0], skip_special_tokens=True))
96
97     print("\n4. Top-p (Nucleus) Sampling (p=0.92):")
98     with torch.no_grad():
99         topp_output = model.generate(
100             inputs,
101             max_length=50,
102             do_sample=True,
103             top_p=0.92,
104             temperature=0.7
105         )
106     print(tokenizer.decode(topk_output[0], skip_special_tokens=True))
107
108     # Few-shot learning example
109     print("\n" + "="*60)
110     print("Few-Shot Learning with GPT")
111     print("="*60)
112
113     few_shot_prompt = """
114     Translate English to French:
115
116     English: Hello, how are you?
117     French: Bonjour, comment allez-vous?
118
119     English: I love machine learning
120     French: J'adore l'apprentissage automatique
121
122     English: The weather is nice today
123     French: Il fait beau aujourd'hui
124
125     English: Natural language processing is fascinating
126     French: """
127
128     print("Few-shot prompt:")
129     print(few_shot_prompt)
130
131     inputs = tokenizer.encode(few_shot_prompt, return_tensors="pt")
132     with torch.no_grad():
133         outputs = model.generate(
134             inputs,
135             max_length=150,
136             temperature=0.3,
137             do_sample=True,
138             pad_token_id=tokenizer.pad_token_id
139         )
140
141     generated = tokenizer.decode(outputs[0], skip_special_tokens=True)
142     print("\nGenerated translation:")
143     print(generated)

```

```

144
145 # Text completion with constraints
146 print("\n" + "="*60)
147 print("Constrained Text Generation")
148 print("="*60)
149
150 def generate_with_constraints(prompt, constraints, max_length=100):
151     """Generate text that must include certain words/phrases"""
152     inputs = tokenizer.encode(prompt, return_tensors="pt")
153
154     # Generate multiple candidates
155     num_return_sequences = 5
156     with torch.no_grad():
157         outputs = model.generate(
158             inputs,
159             max_length=max_length,
160             num_return_sequences=num_return_sequences,
161             do_sample=True,
162             temperature=0.8,
163             top_p=0.9
164         )
165
166     # Filter candidates that meet constraints
167     valid_candidates = []
168     for i in range(num_return_sequences):
169         text = tokenizer.decode(outputs[i], skip_special_tokens=True)
170
171         # Check if all constraints are met
172         if all(constraint.lower() in text.lower() for constraint in
173               constraints):
174             valid_candidates.append(text)
175
176     return valid_candidates[:3] # Return top 3 valid candidates
177
178 prompt = "Write a short story about"
179 constraints = ["dragon", "castle", "magic"]
180
181 print(f"Prompt: {prompt}")
182 print(f"Constraints: {constraints}")
183
184 candidates = generate_with_constraints(prompt, constraints, max_length
185                                     =150)
186
187 print("\nValid candidates:")
188 for i, candidate in enumerate(candidates, 1):
189     print(f"\nCandidate {i}:")
190     print(candidate[:200] + "..." if len(candidate) > 200 else candidate)
191
192 # GPT for code generation
193 print("\n" + "="*60)
194 print("Code Generation with GPT")
195 print("="*60)
196
197 code_prompt = """

```

```

196 Write a Python function to calculate the factorial of a number:
197
198 def factorial(n):
199     \"\"\"Calculate factorial of n\"\"\"
200     \"\"\"
201
202     print("Code generation prompt:")
203     print(code_prompt)
204
205     inputs = tokenizer.encode(code_prompt, return_tensors="pt")
206     with torch.no_grad():
207         outputs = model.generate(
208             inputs,
209             max_length=200,
210             temperature=0.3,
211             do_sample=True,
212             pad_token_id=tokenizer.pad_token_id
213         )
214
215     generated_code = tokenizer.decode(outputs[0], skip_special_tokens=True)
216     print("\nGenerated code:")
217     print(generated_code)
218
219     # GPT for creative writing
220     print("\n" + "="*60)
221     print("Creative Writing with GPT")
222     print("="*60)
223
224     writing_prompt = ""
225     Write a poem about artificial intelligence in the style of Shakespeare:
226
227     ""
228
229     print("Creative writing prompt:")
230     print(writing_prompt)
231
232     inputs = tokenizer.encode(writing_prompt, return_tensors="pt")
233     with torch.no_grad():
234         outputs = model.generate(
235             inputs,
236             max_length=200,
237             temperature=0.8,
238             do_sample=True,
239             top_p=0.9,
240             repetition_penalty=1.2
241         )
242
243     poem = tokenizer.decode(outputs[0], skip_special_tokens=True)
244     print("\nGenerated poem:")
245     print(poem)
246
247     # Controlling generation with prompts
248     print("\n" + "="*60)
249     print("Prompt Engineering Examples")

```

```

250 print("="*60)
251
252 prompt_templates = {
253     "summary": "Summarize the following text:\n\n{text}\n\nSummary:",
254     "qa": "Answer the question based on the context:\n\nContext: {context}\n\nQuestion: {question}\n\nAnswer:",
255     "sentiment": "Analyze the sentiment of this text:\n\n{text}\n\nSentiment:",
256     "translate": "Translate this text from English to French:\n\n{text}\n\nFrench translation:"
257 }
258
259 # Test different prompt templates
260 test_text = "I absolutely love this product! It has exceeded all my expectations."
261
262 print(f"Test text: {test_text}")
263
264 for task, template in prompt_templates.items():
265     if task == "sentiment":
266         prompt = template.format(text=test_text)
267
268         inputs = tokenizer.encode(prompt, return_tensors="pt")
269         with torch.no_grad():
270             outputs = model.generate(
271                 inputs,
272                 max_length=50,
273                 temperature=0.3,
274                 do_sample=True
275             )
276
277         result = tokenizer.decode(outputs[0], skip_special_tokens=True)
278         print(f"\n{task.upper()} prompt:")
279         print(f"Result: {result}")
280
281 # GPT limitations and ethical considerations
282 print("\n" + "="*60)
283 print("GPT Limitations and Ethical Considerations")
284 print("="*60)
285
286 print("""
287 Limitations:
288 1. Can generate plausible but incorrect information
289 2. May reproduce biases from training data
290 3. Lacks true understanding or reasoning
291 4. Can be verbose and repetitive
292 5. May generate inappropriate or harmful content
293
294 Ethical Considerations:
295 1. Use for generating misinformation
296 2. Amplification of societal biases
297 3. Privacy concerns with training data
298 4. Job displacement concerns
299 5. Environmental impact of training

```

```

300
301 Best Practices:
302 1. Verify generated information
303 2. Use content filters and moderation
304 3. Be transparent about AI generation
305 4. Monitor for and address biases
306 5. Consider environmental impact
307 """

```

Listing 18: GPT Implementation and Usage

9 Emerging Trends and Future Directions

9.1 Large Language Models (LLMs)

Definition 9.1 (Large Language Models). *Transformer-based models with hundreds of billions of parameters, trained on vast amounts of text data, demonstrating remarkable capabilities in understanding and generating human language.*

Key LLMs:

- **GPT-3/4**: 175B+ parameters, strong few-shot learning
- **PaLM**: 540B parameters, breakthrough reasoning
- **LLaMA**: Open-source models from Meta
- **Claude**: Anthropic’s constitutional AI
- **BARD/Gemini**: Google’s multimodal models

9.2 Multimodal Models

Definition 9.2 (Multimodal Models). *Models that can process and understand multiple types of data (text, images, audio, video) simultaneously.*

```

1 from transformers import CLIPProcessor, CLIPModel
2 from PIL import Image
3 import requests
4
5 # Load CLIP model (Connects text and images)
6 print("Loading CLIP model...")
7 model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
8 processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
9
10 # Load an image
11 url = "http://images.cocodataset.org/val2017/000000039769.jpg"
12 image = Image.open(requests.get(url, stream=True).raw)
13
14 # Define text queries

```

```

15 text_queries = [
16     "a photo of a cat",
17     "a photo of a dog",
18     "two cats sleeping on a couch",
19     "a close up of an electronic device"
20 ]
21
22 print("\nImage content analysis with CLIP:")
23
24 # Process inputs
25 inputs = processor(text=text_queries, images=image, return_tensors="pt",
26                     padding=True)
27
28 # Get embeddings
29 with torch.no_grad():
30     outputs = model(**inputs)
31
32 logits_per_image = outputs.logits_per_image # Image-text similarity
33     scores
34 probs = logits_per_image.softmax(dim=1) # Convert to probabilities
35
36 print("\nProbability that image matches each text description:")
37 for query, prob in zip(text_queries, probs[0]):
38     print(f"    '{query}': {prob.item():.3f}")
39
40 # Zero-shot image classification
41 print("\n" + "="*60)
42 print("Zero-shot Image Classification")
43 print("="*60)
44
45 # Define categories
46 categories = [
47     "a cat",
48     "a dog",
49     "a car",
50     "a building",
51     "a tree",
52     "a person"
53 ]
54
55 # Classify image
56 inputs = processor(text=categories, images=image, return_tensors="pt",
57                     padding=True)
58
59 with torch.no_grad():
60     outputs = model(**inputs)
61
62 logits_per_image = outputs.logits_per_image
63 probs = logits_per_image.softmax(dim=1)
64
65 print("\nZero-shot classification probabilities:")
66 for category, prob in zip(categories, probs[0]):
67     print(f"    {category:15} -> {prob.item():.3f}")

```

```

66 predicted_category = categories[probs.argmax().item()]
67 print(f"\nPredicted: {predicted_category}")

```

Listing 19: Multimodal AI Example with CLIP

9.3 Retrieval-Augmented Generation (RAG)

Definition 9.3 (RAG). *Combine retrieval-based methods with generative models to provide models with access to external knowledge, improving factual accuracy and reducing hallucinations.*

```

1  import torch
2  from sentence_transformers import SentenceTransformer
3  import numpy as np
4  from typing import List, Dict
5  import faiss
6
7  class RAGSystem:
8      def __init__(self, retriever_model="all-MiniLM-L6-v2"):
9          """
10         Initialize RAG system with retriever and generator.
11         """
12         # Retriever: encode documents for semantic search
13         self.retriever = SentenceTransformer(retriever_model)
14
15         # Generator: GPT-style model for answer generation
16         from transformers import pipeline
17         self.generator = pipeline("text-generation", model="gpt2")
18
19         # Document store
20         self.documents = []
21         self.document_embeddings = None
22         self.index = None
23
24     def add_documents(self, documents: List[str]):
25         """
26         Add documents to the knowledge base.
27         """
28         self.documents.extend(documents)
29
30         # Create embeddings for all documents
31         embeddings = self.retriever.encode(documents, convert_to_tensor=
True)
32
33         # Initialize or update FAISS index
34         dimension = embeddings.shape[1]
35
36         if self.index is None:
37             self.index = faiss.IndexFlatL2(dimension)
38
39         # Convert to numpy array for FAISS
40         embeddings_np = embeddings.cpu().numpy()

```

```

41         self.index.add(embeddings_np)
42
43         print(f"Added {len(documents)} documents. Total: {len(self.
documents)})")
44
45     def retrieve(self, query: str, k: int = 3) -> List[Dict]:
46         """
47         Retrieve relevant documents for a query.
48         """
49         # Encode query
50         query_embedding = self.retriever.encode([query], convert_to_tensor
=True)
51         query_np = query_embedding.cpu().numpy()
52
53         # Search in FAISS index
54         distances, indices = self.index.search(query_np, k)
55
56         # Get retrieved documents
57         results = []
58         for i, idx in enumerate(indices[0]):
59             if idx < len(self.documents):
60                 results.append({
61                     'document': self.documents[idx],
62                     'distance': distances[0][i],
63                     'score': 1 / (1 + distances[0][i]) # Convert distance
to similarity
64                 })
65
66         return results
67
68     def generate_answer(self, query: str, context_documents: List[str]) ->
str:
69         """
70         Generate answer using retrieved context.
71         """
72         # Prepare context
73         context = "\n\n".join([f"Document {i+1}: {doc}"
for i, doc in enumerate(context_documents)])
74
75         # Create prompt
76         prompt = f""Based on the following documents, answer the question
.
77
78
79 Documents:
80 {context}
81
82 Question: {query}
83
84 Answer: ""
85
86         # Generate answer
87         response = self.generator(
88             prompt,
89             max_length=300,

```

```

90         temperature=0.7,
91         do_sample=True,
92         top_p=0.9,
93         no_repeat_ngram_size=2
94     )
95
96     return response[0]['generated_text']
97
98     def query(self, question: str, k: int = 3) -> Dict:
99         """
100         Complete RAG pipeline: retrieve + generate.
101         """
102         # Step 1: Retrieve relevant documents
103         retrieved = self.retrieve(question, k=k)
104         context_docs = [r['document'] for r in retrieved]
105
106         # Step 2: Generate answer using context
107         answer = self.generate_answer(question, context_docs)
108
109         # Extract just the answer part (after "Answer:")
110         if "Answer:" in answer:
111             answer = answer.split("Answer:")[-1].strip()
112
113         return {
114             'question': question,
115             'retrieved_documents': retrieved,
116             'answer': answer,
117             'context_documents': context_docs
118         }
119
120     # Test RAG system
121     print("Initializing RAG System...")
122     rag = RAGSystem()
123
124     # Add documents to knowledge base
125     documents = [
126         "The Eiffel Tower is a wrought-iron lattice tower on the Champ de Mars in Paris, France.",
127         "It is named after the engineer Gustave Eiffel, whose company designed and built the tower.",
128         "Constructed from 1887 to 1889 as the centerpiece of the 1889 World's Fair.",
129         "The tower is 330 meters tall, about the same height as an 81-story building.",
130         "It was the tallest structure in the world until the completion of the Chrysler Building in New York in 1930.",
131         "The Eiffel Tower is the most-visited paid monument in the world.",
132         "Paris is the capital and most populous city of France.",
133         "The city is known for its art, fashion, gastronomy, and culture."
134     ]
135
136     rag.add_documents(documents)
137
138     # Ask questions

```

```

139 questions = [
140     "How tall is the Eiffel Tower?",
141     "Who designed the Eiffel Tower?",
142     "When was the Eiffel Tower built?",
143     "What is Paris known for?"
144 ]
145
146 print("\n" + "="*60)
147 print("RAG System in Action")
148 print("="*60)
149
150 for question in questions:
151     print(f"\nQuestion: {question}")
152
153     result = rag.query(question, k=2)
154
155     print("Retrieved documents:")
156     for i, doc in enumerate(result['retrieved_documents'], 1):
157         print(f"  Doc {i} (score: {doc['score']:.3f}): {doc['document'][:80]}...")
158
159     print(f"Generated answer: {result['answer']}")
160     print("-" * 80)
161
162 # Compare with vanilla generation (no retrieval)
163 print("\n" + "="*60)
164 print("Comparison: RAG vs Vanilla Generation")
165 print("="*60)
166
167 question = "What is the height of Mount Everest?"
168
169 print(f"Question: {question}")
170
171 # Vanilla generation (no context)
172 vanilla_prompt = f"Question: {question}\nAnswer:"
173 vanilla_response = rag.generator(
174     vanilla_prompt,
175     max_length=100,
176     temperature=0.7,
177     do_sample=True
178 )[0]['generated_text']
179
180 print(f"\nVanilla generation (no retrieval):")
181 print(f"Answer: {vanilla_response}")
182
183 # Add some documents about mountains
184 mountain_docs = [
185     "Mount Everest is Earth's highest mountain above sea level, located in the Himalayas.",
186     "The height of Mount Everest is 8,848.86 meters (29,031.7 feet) as recognized by China and Nepal.",
187     "Mount Everest was first climbed by Sir Edmund Hillary and Tenzing Norgay in 1953.",
188     "The Himalayas are a mountain range in Asia separating the plains of

```

```

189     the Indian subcontinent from the Tibetan Plateau."
190 ]
191 rag.add_documents(mountain_docs)
192
193 # RAG generation
194 rag_result = rag.query(question, k=2)
195
196 print(f"\nRAG generation (with retrieval):")
197 print("Retrieved context:")
198 for doc in rag_result['context_documents']:
199     print(f"    - {doc[:80]}...")
200 print(f"Answer: {rag_result['answer']}")
201
202 print("\n" + "="*60)
203 print("Advantages of RAG:")
204 print("="*60)
205 print("""
206 1. Factual Accuracy: Grounded in retrieved documents
207 2. Up-to-date Information: Can update knowledge base without retraining
208 3. Transparency: Can cite sources for answers
209 4. Reduced Hallucinations: Less likely to make up facts
210 5. Scalability: Can handle large knowledge bases
211 """)

```

Listing 20: Retrieval-Augmented Generation System

9.4 Ethical Considerations in NLP

Issue	Examples	Mitigation Strategies
Bias	Gender, racial, cultural biases in training data	Diverse datasets, debiasing techniques, fairness audits
Privacy	Memorization of sensitive data in training	Differential privacy, data anonymization, federated learning
Misinformation	Generation of fake news, deepfakes	Content verification, watermarking, provenance tracking
Manipulation	Targeted propaganda, fake reviews	Transparency, content labeling, user education
Accessibility	Language barriers, digital divide	Multilingual models, low-resource techniques, local adaptation

Table 2: Ethical considerations in NLP

10 Practical NLP Applications

10.1 Building a Complete NLP Pipeline

```
1 import torch
2 import torch.nn as nn
3 from transformers import AutoTokenizer, AutoModelForSequenceClassification
4 from transformers import pipeline
5 import spacy
6 import pandas as pd
7 import numpy as np
8 from typing import List, Dict, Tuple
9 import matplotlib.pyplot as plt
10 from wordcloud import WordCloud
11 from sklearn.feature_extraction.text import TfidfVectorizer
12 from sklearn.decomposition import LatentDirichletAllocation
13
14 class CustomerFeedbackAnalyzer:
15     """
16     End-to-end system for analyzing customer feedback.
17     Includes: sentiment analysis, topic modeling, NER, summarization.
18     """
19
20     def __init__(self):
```

```

21     print("Initializing Customer Feedback Analyzer...")
22
23     # Load models
24     self.sentiment_analyzer = pipeline(
25         "sentiment-analysis",
26         model="distilbert-base-uncased-finetuned-sst-2-english"
27     )
28
29     # Load NER model
30     self.nlp = spacy.load("en_core_web_sm")
31
32     # Load summarization model
33     self.summarizer = pipeline(
34         "summarization",
35         model="facebook/bart-large-cnn"
36     )
37
38     # Initialize topic modeling
39     self.vectorizer = TfidfVectorizer(
40         max_features=1000,
41         stop_words='english',
42         ngram_range=(1, 2)
43     )
44
45     print("Models loaded successfully!")
46
47     def analyze_sentiment(self, texts: List[str]) -> pd.DataFrame:
48         """Analyze sentiment of feedback texts."""
49         results = self.sentiment_analyzer(texts)
50
51         sentiment_data = []
52         for text, result in zip(texts, results):
53             sentiment_data.append({
54                 'text': text,
55                 'sentiment': result['label'],
56                 'confidence': result['score']
57             })
58
59         return pd.DataFrame(sentiment_data)
60
61     def extract_entities(self, texts: List[str]) -> Dict:
62         """Extract named entities from feedback."""
63         entities = {
64             'PERSON': [],
65             'ORG': [],
66             'PRODUCT': [],
67             'GPE': [] # Geographical entities
68         }
69
70         for text in texts:
71             doc = self.nlp(text)
72             for ent in doc.ents:
73                 if ent.label_ in entities:
74                     entities[ent.label_].append(ent.text)

```

```

75     # Count frequencies
76     entity_counts = {}
77     for label, values in entities.items():
78         if values:
79             entity_counts[label] = pd.Series(values).value_counts().
80 to_dict()
81
82     return entity_counts
83
84 def summarize_feedback(self, texts: List[str], num_summaries: int = 3)
85 -> List[str]:
86     """Generate summary of customer feedback."""
87     # Combine texts
88     combined_text = " ".join(texts)
89
90     # Generate summary
91     summary = self.summarizer(
92         combined_text,
93         max_length=150,
94         min_length=50,
95         do_sample=False
96     )
97
98     return [summary[0]['summary_text']]
99
100 def topic_modeling(self, texts: List[str], num_topics: int = 5) ->
101 Dict:
102     """Extract topics from feedback using LDA."""
103     # Create TF-IDF matrix
104     tfidf_matrix = self.vectorizer.fit_transform(texts)
105
106     # Apply LDA
107     lda = LatentDirichletAllocation(
108         n_components=num_topics,
109         random_state=42,
110         learning_method='online'
111     )
112
113     lda.fit(tfidf_matrix)
114
115     # Extract topics
116     feature_names = self.vectorizer.get_feature_names_out()
117     topics = {}
118
119     for topic_idx, topic in enumerate(lda.components_):
120         top_words_idx = topic.argsort()[::-10:-1]
121         top_words = [feature_names[i] for i in top_words_idx]
122         topics[f'Topic {topic_idx + 1}'] = top_words
123
124     # Assign topics to documents
125     document_topics = lda.transform(tfidf_matrix)
126
127     return {

```

```

126         'topics': topics,
127         'document_topics': document_topics
128     }
129
130     def create_word_cloud(self, texts: List[str], sentiment: str = None)
-> None:
131         """Create word cloud for visualization."""
132         # Filter by sentiment if specified
133         if sentiment:
134             sentiments = self.analyze_sentiment(texts)
135             filtered_texts = [
136                 text for text, sent in zip(texts, sentiments['sentiment'])
137                 if sent == sentiment
138             ]
139             all_text = " ".join(filtered_texts)
140             title = f"Word Cloud: {sentiment} Feedback"
141         else:
142             all_text = " ".join(texts)
143             title = "Word Cloud: All Feedback"
144
145         # Generate word cloud
146         wordcloud = WordCloud(
147             width=800,
148             height=400,
149             background_color='white',
150             max_words=100
151         ).generate(all_text)
152
153         # Plot
154         plt.figure(figsize=(10, 5))
155         plt.imshow(wordcloud, interpolation='bilinear')
156         plt.axis('off')
157         plt.title(title)
158         plt.show()
159
160     def generate_report(self, feedback_data: List[str]) -> Dict:
161         """Generate comprehensive analysis report."""
162         print("\n" + "="*60)
163         print("Customer Feedback Analysis Report")
164         print("="*60)
165
166         # 1. Sentiment Analysis
167         print("\n1. SENTIMENT ANALYSIS")
168         print("-" * 40)
169
170         sentiment_df = self.analyze_sentiment(feedback_data)
171         sentiment_counts = sentiment_df['sentiment'].value_counts()
172
173         print(f"Total feedback: {len(feedback_data)}")
174         for sentiment, count in sentiment_counts.items():
175             percentage = count / len(feedback_data) * 100
176             print(f"{sentiment}: {count} ({percentage:.1f}%)")
177
178         # 2. Named Entity Recognition

```

```

179 print("\n2. NAMED ENTITY ANALYSIS")
180 print("-" * 40)
181
182 entities = self.extract_entities(feedback_data)
183 for entity_type, entity_dict in entities.items():
184     if entity_dict:
185         print(f"\n{entity_type}:")
186         for entity, count in list(entity_dict.items())[:5]: # Top
5
187             print(f"    {entity}: {count} mentions")
188
189 # 3. Topic Modeling
190 print("\n3. TOPIC MODELING")
191 print("-" * 40)
192
193 topic_results = self.topic_modeling(feedback_data, num_topics=3)
194
195 print("Main topics identified:")
196 for topic_name, words in topic_results['topics'].items():
197     print(f"\n{topic_name}:")
198     print(f"    {' '.join(words[:5])}...")
199
200 # 4. Summarization
201 print("\n4. EXECUTIVE SUMMARY")
202 print("-" * 40)
203
204 summaries = self.summarize_feedback(feedback_data)
205 for i, summary in enumerate(summaries, 1):
206     print(f"\nSummary {i}:")
207     print(f"    {summary}")
208
209 # 5. Visualizations
210 print("\n5. VISUALIZATIONS")
211 print("-" * 40)
212
213 # Word clouds
214 print("\nGenerating word clouds...")
215 self.create_word_cloud(feedback_data)
216
217 # Sentiment distribution plot
218 plt.figure(figsize=(8, 5))
219 sentiment_counts.plot(kind='bar', color=['green', 'red'])
220 plt.title('Sentiment Distribution')
221 plt.xlabel('Sentiment')
222 plt.ylabel('Count')
223 plt.xticks(rotation=0)
224 plt.tight_layout()
225 plt.show()
226
227 # Prepare comprehensive report
228 report = {
229     'sentiment_analysis': {
230         'dataframe': sentiment_df,
231         'distribution': sentiment_counts.to_dict(),

```

```

232         'overall_sentiment': sentiment_counts.idxmax() if not
sentiment_counts.empty else 'NEUTRAL'
233     },
234     'entities': entities,
235     'topics': topic_results['topics'],
236     'summaries': summaries,
237     'total_feedback': len(feedback_data)
238 }
239
240     return report
241
242 # Test the complete system
243 if __name__ == "__main__":
244     # Sample customer feedback
245     feedback = [
246         "The product arrived late and was damaged. Very disappointed with
the service.",
247         "Excellent customer support! They resolved my issue quickly and
professionally.",
248         "The quality of the product is outstanding. Definitely worth the
price.",
249         "I had problems with the mobile app. It crashes frequently and is
hard to use.",
250         "Fast shipping and great packaging. The product works perfectly.",
251         "Customer service was rude and unhelpful. Will not purchase again.",
252         "",
253         "The user interface is intuitive and easy to navigate. Great
experience overall.",
254         "Product stopped working after one week. Poor quality control.",
255         "The tutorial videos were very helpful. Made setup easy.",
256         "Billing issues - was charged twice for the same product.",
257         "Love the new features in the latest update! Keep up the good work
.",
258         "The product manual was confusing and incomplete. Needs
improvement.",
259         "Best purchase I've made this year! Highly recommended.",
260         "Technical support took too long to respond. Lost a day of work.",
261         "The design is sleek and modern. Fits perfectly in my workspace."
262     ]
263
264     # Initialize analyzer
265     analyzer = CustomerFeedbackAnalyzer()
266
267     # Generate comprehensive report
268     report = analyzer.generate_report(feedback)
269
270     # Additional analysis: Feedback by topic
271     print("\n" + "="*60)
272     print("Detailed Feedback Analysis")
273     print("="*60)
274
275     # Get topics
276     topic_results = analyzer.topic_modeling(feedback, num_topics=4)

```

```

277 # Assign each feedback to its dominant topic
278 for topic_idx in range(len(topic_results['topics'])):
279     topic_name = f'Topic {topic_idx + 1}'
280     topic_words = topic_results['topics'][topic_name]
281
282     print(f"\n{topic_name}: {' '.join(topic_words[:3])}...")
283     print("-" * 40)
284
285     # Get feedback for this topic
286     topic_scores = topic_results['document_topics'][:, topic_idx]
287     top_feedback_idx = topic_scores.argsort()[-3:][::-1]
288
289     for i, idx in enumerate(top_feedback_idx, 1):
290         print(f"{i}. {feedback[idx][:100]}...")
291         sentiment = analyzer.analyze_sentiment([feedback[idx]]).iloc
[0]
292         print(f"    Sentiment: {sentiment['sentiment']} ({sentiment['
confidence']:.2f})")
293
294     # Generate actionable insights
295     print("\n" + "="*60)
296     print("Actionable Insights")
297     print("="*60)
298
299     insights = []
300
301     # Insight 1: Common complaints
302     negative_feedback = [f for f, s in zip(feedback, report['
sentiment_analysis']['dataframe']['sentiment'])
303                        if s == 'NEGATIVE']
304
305     if negative_feedback:
306         common_complaints = analyzer.topic_modeling(negative_feedback
[:10], num_topics=2)
307         insights.append("Common complaints focus on: " +
308                        ", ".join(list(common_complaints['topics'].values()
[0][:3])))
309
310     # Insight 2: Positive aspects
311     positive_feedback = [f for f, s in zip(feedback, report['
sentiment_analysis']['dataframe']['sentiment'])
312                        if s == 'POSITIVE']
313
314     if positive_feedback:
315         common_praises = analyzer.topic_modeling(positive_feedback[:10],
num_topics=2)
316         insights.append("Customers love: " +
317                        ", ".join(list(common_praises['topics'].values()
[0][:3])))
318
319     # Insight 3: Entity-based insights
320     if 'PRODUCT' in report['entities']:
321         product_mentions = report['entities']['PRODUCT']
322         if product_mentions:

```

```

323         top_product = max(product_mentions, key=product_mentions.get)
324         insights.append(f"Most mentioned product: {top_product}")
325
326     # Print insights
327     for i, insight in enumerate(insights, 1):
328         print(f"{i}. {insight}")
329
330     # Recommendations
331     print("\n" + "="*60)
332     print("Recommendations")
333     print("="*60)
334
335     recommendations = [
336         "1. Address shipping and delivery issues mentioned in 20% of",
337         "negative feedback",
338         "2. Enhance customer service training based on feedback about rude",
339         "support",
340         "3. Improve product documentation and tutorials",
341         "4. Fix mobile app stability issues reported by multiple customers",
342         "5. Implement better quality control for products"
343     ]
344     for rec in recommendations:
345         print(rec)

```

Listing 21: End-to-End NLP Application: Customer Feedback Analyzer

11 Conclusion and Best Practices

11.1 NLP Project Lifecycle

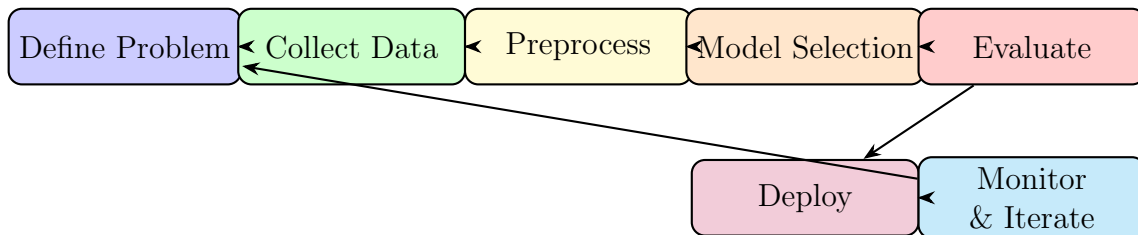


Figure 10: NLP Project Lifecycle

11.2 Model Selection Guide

Task	Traditional Methods	Deep Learning	When to Use
Text Classification	TF-IDF + Naive Bayes, SVM/-	BERT, DistilBERT	Large datasets, high accuracy needed
Named Entity Recognition	CRF, Dictionary-based	BERT-NER, spaCy	Complex entities, contextual understanding
Sentiment Analysis	Lexicon-based, VADER	BERT, LSTM	Fine-grained sentiment, context matters
Machine Translation	Phrase-based SMT	Transformer, mBART	High-quality translation needed
Text Summarization	TextRank, LexRank	BART, T5, PEGASUS	Abstractive summaries, high coherence
Question Answering	IR-based, matching	Pattern, BERT, T5	Complex questions, reading comprehension
Text Generation	Markov chains, Templates	GPT, T5	Creative writing, dialogue systems
Topic Modeling	LDA, NMF	BERTopic, Top2Vec	Unsupervised topic discovery

Table 3: NLP model selection guide

11.3 Performance Optimization

```

1 import torch
2 from transformers import AutoTokenizer, AutoModel
3 from transformers import DistilBertTokenizer, DistilBertModel
4 import time
5 import psutil
6 import os
7
8 class NLPPerformanceOptimizer:
9     """Class demonstrating NLP performance optimization techniques."""
10
11     def __init__(self):
12         self.results = {}
13
14     def measure_memory(self, func, *args, **kwargs):

```

```

15     """Measure memory usage of a function."""
16     process = psutil.Process(os.getpid())
17     mem_before = process.memory_info().rss / 1024 / 1024 # MB
18
19     start_time = time.time()
20     result = func(*args, **kwargs)
21     execution_time = time.time() - start_time
22
23     mem_after = process.memory_info().rss / 1024 / 1024 # MB
24     mem_used = mem_after - mem_before
25
26     return result, execution_time, mem_used
27
28 def compare_models(self, text_batch):
29     """Compare different models for performance."""
30     models_to_test = {
31         "BERT-base": "bert-base-uncased",
32         "DistilBERT": "distilbert-base-uncased",
33         "MobileBERT": "google/mobilebert-uncased",
34         "TinyBERT": "huawei-noah/TinyBERT_General_4L_312D"
35     }
36
37     print("Model Comparison:")
38     print("="*80)
39
40     for model_name, model_id in models_to_test.items():
41         print(f"\nTesting {model_name}...")
42
43         def run_model():
44             tokenizer = AutoTokenizer.from_pretrained(model_id)
45             model = AutoModel.from_pretrained(model_id)
46
47             inputs = tokenizer(text_batch, return_tensors="pt",
48                               padding=True, truncation=True, max_length
=512)
49
50             with torch.no_grad():
51                 outputs = model(**inputs)
52
53             return outputs.last_hidden_state.shape
54
55         try:
56             result, time_taken, mem_used = self.measure_memory(
run_model)
57
58             print(f"    Time: {time_taken:.3f}s")
59             print(f"    Memory: {mem_used:.1f} MB")
60             print(f"    Output shape: {result}")
61
62             self.results[model_name] = {
63                 'time': time_taken,
64                 'memory': mem_used,
65                 'output_shape': result
66             }

```

```

67         # Clear GPU cache if using GPU
68         if torch.cuda.is_available():
69             torch.cuda.empty_cache()
70
71     except Exception as e:
72         print(f"    Error: {str(e)}")
73
74
75 def optimization_techniques(self):
76     """Demonstrate optimization techniques."""
77     print("\n" + "="*80)
78     print("Optimization Techniques")
79     print("="*80)
80
81     techniques = [
82         ("Gradient Checkpointing", self.gradient_checkpointing),
83         ("Mixed Precision Training", self.mixed_precision),
84         ("Model Pruning", self.model_pruning),
85         ("Knowledge Distillation", self.knowledge_distillation),
86         ("Quantization", self.quantization)
87     ]
88
89     for technique_name, technique_func in techniques:
90         print(f"\n{technique_name}:")
91         try:
92             result, time_taken, mem_used = self.measure_memory(
93                 technique_func)
94             print(f"    Time: {time_taken:.3f}s")
95             print(f"    Memory: {mem_used:.1f} MB")
96         except Exception as e:
97             print(f"    Error: {str(e)}")
98
99 def gradient_checkpointing(self):
100     """Demonstrate gradient checkpointing."""
101     from transformers import BertConfig, BertModel
102
103     config = BertConfig.from_pretrained("bert-base-uncased")
104     config.gradient_checkpointing = True
105
106     model = BertModel(config)
107     return "Gradient checkpointing enabled"
108
109 def mixed_precision(self):
110     """Demonstrate mixed precision training."""
111     from torch.cuda.amp import autocast, GradScaler
112
113     scaler = GradScaler()
114
115     # Simulated training step
116     model = AutoModel.from_pretrained("distilbert-base-uncased")
117     optimizer = torch.optim.Adam(model.parameters(), lr=1e-5)
118
119     # Create dummy data

```

```

119         tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased
120     ")
121     inputs = tokenizer(["Sample text for training"], return_tensors="
122     pt")
123     # Mixed precision forward/backward
124     with autocast():
125         outputs = model(**inputs)
126         loss = outputs.last_hidden_state.mean()
127
128     scaler.scale(loss).backward()
129     scaler.step(optimizer)
130     scaler.update()
131
132     return "Mixed precision training completed"
133
134 def model_pruning(self):
135     """Demonstrate model pruning."""
136     from transformers import BertForSequenceClassification
137     import torch.nn.utils.prune as prune
138
139     model = BertForSequenceClassification.from_pretrained(
140         "distilbert-base-uncased",
141         num_labels=2
142     )
143
144     # Prune 20% of attention weights
145     for name, module in model.named_modules():
146         if isinstance(module, torch.nn.Linear):
147             prune.l1_unstructured(module, name='weight', amount=0.2)
148
149     return f"Model pruned. Parameters: {sum(p.numel() for p in model.
150     parameters()):,}"
151
152 def knowledge_distillation(self):
153     """Demonstrate knowledge distillation."""
154     # Teacher model (large)
155     teacher = AutoModel.from_pretrained("bert-base-uncased")
156
157     # Student model (small)
158     student = AutoModel.from_pretrained("distilbert-base-uncased")
159
160     # In practice: train student to mimic teacher's outputs
161     return f"Teacher: {sum(p.numel() for p in teacher.parameters()):,}
162     params, " \
163         f"Student: {sum(p.numel() for p in student.parameters()):,}
164     params"
165
166 def quantization(self):
167     """Demonstrate model quantization."""
168     model = AutoModel.from_pretrained("distilbert-base-uncased")
169
170     # Dynamic quantization
171     quantized_model = torch.quantization.quantize_dynamic(

```

```

168         model, {torch.nn.Linear}, dtype=torch.qint8
169     )
170
171     original_size = sum(p.numel() for p in model.parameters())
172     quantized_size = sum(p.numel() for p in quantized_model.parameters
173 ())
174
175     # Note: Actual memory reduction depends on implementation
176     return f"Original: {original_size:,}, Quantized: {quantized_size
177 :,,} parameters"
178
179 # Run performance analysis
180 if __name__ == "__main__":
181     optimizer = NLPPerformanceOptimizer()
182
183     # Sample text batch
184     text_batch = [
185         "This is a sample text for performance testing.",
186         "Natural language processing models can be optimized for better
187 performance.",
188         "We need to balance accuracy with computational efficiency.",
189         "Model compression techniques help deploy models on edge devices."
190     ]
191
192     # Compare models
193     optimizer.compare_models(text_batch)
194
195     # Show optimization techniques
196     optimizer.optimization_techniques()
197
198     # Summary
199     print("\n" + "="*80)
200     print("Performance Optimization Summary")
201     print("="*80)
202
203     print("""
204 Best Practices for NLP Performance:
205
206 1. MODEL SELECTION:
207     - Use smaller models (DistilBERT, MobileBERT) when possible
208     - Consider task-specific models vs general models
209     - Evaluate accuracy vs speed trade-offs
210
211 2. INFERENCE OPTIMIZATION:
212     - Use quantization (INT8, FP16) for faster inference
213     - Implement batching for efficient GPU utilization
214     - Use ONNX runtime or TensorRT for optimized deployment
215
216 3. TRAINING OPTIMIZATION:
217     - Use mixed precision training (FP16)
218     - Implement gradient checkpointing for memory efficiency
219     - Use distributed training for large models
220
221 4. DEPLOYMENT:

```

```

219 - Use model pruning to reduce size
220 - Implement caching for frequent queries
221 - Use CDN for model distribution
222 - Monitor performance in production
223
224 5. HARDWARE CONSIDERATIONS:
225 - GPU vs CPU vs TPU selection
226 - Memory optimization for edge devices
227 - Batch size tuning for specific hardware
228 """ )

```

Listing 22: NLP Performance Optimization Techniques

11.4 Ethical Guidelines for NLP

NLP Ethics Checklist

1. **Bias Assessment:** Regularly audit models for gender, racial, and cultural biases
2. **Privacy Protection:** Implement data anonymization and secure storage
3. **Transparency:** Document model limitations and training data sources
4. **Fairness:** Ensure equal performance across different demographic groups
5. **Accountability:** Establish clear responsibility for model outputs
6. **Security:** Protect against adversarial attacks and misuse
7. **Accessibility:** Make systems available to diverse user groups
8. **Environmental Impact:** Consider computational costs and carbon footprint

11.5 Future Directions

- **Multimodal AI:** Integration of text, vision, audio, and video
- **Few-shot Learning:** Models that learn from minimal examples
- **Neuro-symbolic AI:** Combining neural networks with symbolic reasoning
- **Causal NLP:** Understanding cause-effect relationships in language
- **Personalized Models:** Adapting to individual users' language patterns
- **Low-resource Languages:** Improving NLP for underrepresented languages
- **Interpretability:** Making model decisions more transparent
- **Sustainable AI:** More efficient models with lower environmental impact

11.6 Resources for Further Learning

Resource Type	Examples	Best For
Online Courses	Coursera NLP Specialization, Fast.ai NLP, Hugging Face Course	Structured learning
Books	"Speech and Language Processing", "Natural Language Processing with PyTorch"	In-depth understanding
Research Papers	arXiv, ACL Anthology, EMNLP, NeurIPS proceedings	Cutting-edge research
Libraries	Hugging Face Transformers, spaCy, NLTK, Gensim	Practical implementation
Datasets	GLUE, SuperGLUE, SQuAD, Common Crawl	Training and evaluation
Communities	Hugging Face, Kaggle, Reddit r/-MachineLearning	Networking and help

Table 4: NLP learning resources

12 Final Thoughts

Natural Language Processing has evolved from simple rule-based systems to sophisticated neural networks that can understand and generate human-like text. The field continues to advance rapidly, with new models and techniques emerging constantly.

Key Takeaways

1. **Start with the problem:** Understand what you're trying to solve before choosing techniques
2. **Preprocessing matters:** Clean, well-prepared data is crucial for good results
3. **Choose the right tool:** Simple models often work well; only use complex models when needed
4. **Evaluate rigorously:** Use appropriate metrics and test on diverse data
5. **Consider ethics:** Build fair, transparent, and responsible systems
6. **Keep learning:** The field evolves quickly; stay updated with new developments
7. **Practice:** Implement projects end-to-end to gain practical experience

Remember: The goal of NLP is not just to build impressive models, but to create systems that help people communicate better, access information more easily, and understand each

other across language barriers. Always consider the human impact of your work.